

**Document Number:** P2179R0  
**Date:** 2020-06-02  
**Audience:** SG16  
**Reply-to:** Tom Honermann <tom@honermann.net>

## SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>. This paper contains a snapshot of select meeting summaries from that repository.

- [January 8th, 2020](#)
- [January 22nd, 2020](#)
- [February 5th, 2020](#)
- [February 26th, 2020](#)
- [March 11th, 2020](#)
- [March 25th, 2020](#)
- [April 8th, 2020](#)
- [April 22nd, 2020](#)
- [May 13th, 2020](#)
- [May 27th, 2020](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)

## January 8th, 2020

### Draft agenda:

- LWG issue 3341: `basic_regex` range constructor: Missing requirements for iterator types
  - <https://cplusplus.github.io/LWG/issue3341>
  - Billy O'Neal copied SG16 on this issue; see <https://lists.isocpp.org/sg16/2019/12/0990.php>
  - What should be the proposed resolution?
- P1949: C++ Identifier Syntax using Unicode Standard Annex 31:
  - <https://wg21.link/p1949>
  - Which UAX #31 requirements do we intend to satisfy (see section 2)?
  - Which UAX #31 specific character adjustments do we want (see section 2.4)?
  - Which UAX #31 NFKC modifications do we want (see section 5.1)?

### Attendees:

- Amanda Kornoushenko
- David Wendt
- JeanHeyd Meneide
- Peter Bindels
- Steve Downey
- Tom Honermann
- Zach Laine

### Meeting summary:

- LWG issue 3341: `basic_regex` range constructor: Missing requirements for iterator types
  - <https://cplusplus.github.io/LWG/issue3341>
  - Tom introduced the topic:

- Billy O'Neal copied SG16 on this issue. His email is available in the SG16 mailing list archives at <https://lists.isocpp.org/sg16/2019/12/0990.php>.
  - What should be the proposed resolution?
- Zach asked why we should be concerned about this issue.
- Tom responded that Billy copied us on it, presumably seeking our input.
- Zach stated that implicit transcoding should not occur; The safest thing to do would be to require `ForwardIterator::value_type` to be exactly `charT`.
- JeanHeyd agreed with the no implicit transcoding stance; that would make `std::regex` even slower!
- Peter chimed in via chat agreeing with an exact type requirement and the following constraint:
  - `std::is_same_t<value_type, decltype(*std::declval<ForwardIterator>())>`
- Peter asked if we should enable views to be used as inputs.
- Steve stated that would be a more difficult challenge given the recent difficulties faced when attempting to add range constructors for standard containers.
- Zach agreed that adding range support could be difficult and is out of scope for this issue anyway.
- Peter concurred and noted that view support can always be added later.
- Tom observed that if a same type constraint is added, then SFINAE will kick in, but it might be preferred to make it a hard error if the iterator value type doesn't match.
- Zach suggested leaving that for LWG to decide.
- Tom agreed and stated he would respond to Billy's email and LWG with our thoughts.
- P1949: C++ Identifier Syntax using Unicode Standard Annex 31:
  - <https://wg21.link/p1949>
  - <https://github.com/cplusplus/nbballot/issues/28>
  - Tom introduced the topic.
    - In Belfast, EWG did not accept [SG16's recommended resolution for NL029](#) for C++20.
    - Tom volunteered to submit a core issue for C++20 in order to allow us to resolve the concern as a defect, but he doesn't have a PR to propose.
    - Tom is thinking about bailing on submitting that core issue, but thought he would check if SG16 might have consensus on what solution we would want. In particular, if we were to adopt [UAX #31](#), what would be our answers to these questions?
      - Which UAX #31 requirements do we intend to satisfy (see section 2)?
      - Which UAX #31 specific character adjustments do we want (see section 2.4)?
      - Which UAX #31 NFKC modifications we want (see section 5.1)?
  - Zach suggested we skip C++20 and just proceed with addressing this for C++23.
  - Steve provided motivation for dealing with this as a DR; some compilers are just starting to allow extended characters in identifiers. Previously, programmers had to go out of their way to create weird identifiers. Clang has allowed extended characters forever (since Clang 3.3 or so), gcc support was added for gcc 10. The window for changing behavior is shrinking.
  - Zach asked if the concern was about breaking existing code given that this isn't the kind of break we usually worry too much about.
  - Tom replied that the breakage could be silent if Unicode normalization affects whether two identifiers match.
  - Steve added that breakage could occur due to excluded characters like the poop emoji. Compilers could provide backward compatibility options; Hyrum's law.
  - Steve continued stating that, if we don't get this nailed down for C++23, we could probably still do it because it probably won't affect that much code.
  - Zach observed that the impact would mostly be due to banning emoji.
  - Steve agreed; emoji is the only case people are likely to notice. Programmers aren't likely to want right-to-left characters in identifiers for example.
  - Which UAX #31 requirements do we intend to satisfy (see section 2)?
    - Tom stated that we need to choose whether to use the `ID_Start`/`ID_Continue` or `XID_Start`/`XID_Continue` properties to define identifier syntax. P1949 suggests using the `XID_Start`/`XID_Continue` variants and doing so is necessary to meet the requirements for [UAX31-R1](#) without defining a profile; though, we'll need a profile to add `_` as a start character.
    - Steve recommended we adopt the `XID` variants and add `_` as a start character. However, this doesn't suffice to guarantee identifier stability.
    - Tom stated that, in order to meet requirement [UAX31-R1a](#), that he thinks we'll need to specify additional characters to exclude. The NL029 NB comment specified a particular range to exclude, but he is not sure if or how that matches UAX #31.
    - Steve corrected Tom's interpretation; that requirement allows opting in to characters that are disallowed by default.
    - Peter stated that section 2.3 explains that some character that are restricted by default are needed in some cases for some scripts.
    - Peter continued stating that he thinks we lack the experience to make choices in this regard and suggested we proceed with more restrictions now and relax them later based on experience and motivation.
    - Tom asked about meeting the requirements for [UAX31-R1b](#); assuming we want to meet that requirement, how would we do so?
    - Steve responded that, given ABI issues, we should commit to meeting this requirement. In practice, that means that, for example, if a future Unicode standard were to remove characters from `XID_CONTINUE`, that we would update our profile to add them back in.
    - Peter asked if the `XID_Start`/`XID_Continue` properties are stable.
    - Zach responded that he understood them to be stable.
    - Steve responded that they are derived properties and are not guaranteed to be stable, but probably will be in practice.
    - [ Editor's note: in later email discussion, Steve offered a correction to this statement: `XID_Start` and `XID_Continue` are guaranteed stable, just not immutable. Immutability is the property that things that are not identifiers remain not identifiers. ]

- Zach mentioned that he wasn't previously aware that UAX #31 had options, but it seems our goal now needs to be to identify the options, select them, and then make sure proposed wording reflects our intent.
- Tom agreed.
- Which UAX #31 specific character adjustments do we want (see section 2.4)?
  - Peter, reviewing section 2.4, stated no observed need for exceptions other than for `_` in the start position; a choice that is already explicitly listed as an option.
- Which UAX #31 NFC modifications we we want (see section 5.1)?
  - Tom stated that we need to figure out how to deal with normalization if we want stable identifiers.
  - Zach provided some background on NFC, NFD, compatibility, comparisons, and conversions.
  - Zach professed support for standardizing on NFC; NFD is not really usable since combining marks don't tend to be represented by themselves in identifiers.
  - Tom asked if standardizing on NFC commits implementors to perform normalization.
  - Steve responded that gcc 10 already emits a warning for identifiers that are written in non-NFC forms in source code.
  - Zach stated that checking for NFC is fast, at least for common cases, so diagnosing is reasonable, but stating that non-NFC identifiers are IFNDR (ill-formed no diagnostic required) is also a possibility.
  - Tom observed that conversions from other character sets like Windows-1252 probably always result in NFC.
  - Zach agreed noting that such conversion is probably done via the compiler's internal encoding.
  - Peter stated that there are other character sets that have combining marks, but none of those are probably supported by compilers.
  - Tom, considering source code that is encoded as UTF-8 in NFD, asked if requiring NFC could be problematic for existing editors and tools.
  - Steve observed that this issue already exists and that tools today already expect NFC.
  - Peter recommended that we make use of non-NFC normalized source code IFNDR and encourage tools to diagnose violations.
  - Zach responded that IFNDR is generally reserved for cases where something can't be reasonably diagnosed; since diagnosis is reasonable here, non-NFC forms should be considered ill-formed.
  - Steve added that compiler implementors can support options to relax NFC checking.
  - Tom noted that this creates a specification issue since, if source encoding is not UTF-8, it needs to be transcoded to NFC, but if it is UTF-8, source code needs to already be in NFC.
  - Zach responded that we don't have to; we just specify the characters that are valid based on `XID_Start/XID_Continue`.
  - Steve added that the NFC check has to be done after conversion from source encoding to internal encoding and that he is unaware of any encoding that does not naturally transcode to NFC.
  - Peter observed that combining diacritics are part of `XID_Continue` and that there are therefore two spellings of café; a 4 code point variant using U+00E9 {LATIN SMALL LETTER E WITH ACUTE} and a 5 code point variant using U+0301 {COMBINING ACUTE ACCENT}.
  - Zach stated that this feature requires the compiler's internal encoding to be Unicode.
  - Tom responded that, since C++11, the internal encoding must already be isomorphic to Unicode.
  - Zach suggested that both forms of café should not be allowed; that NFC should be required, and that use of combining characters should be disallowed in our profile.
  - Steve responded that disallowing all combining characters probably isn't feasible; there aren't precomposed forms of all characters; in NFC, combining characters will still appear, but only when they are actually required.
  - Zach suggested this is a restriction that could be relaxed later.
  - Steve observed that this would make specification of the profile more difficult.
  - Zach agreed and suggested just specifying a list of start and continue characters; this avoids implementors having to do hard things.
  - Peter noticed a problem with that approach; new Unicode characters could not be used unless and until the standard is updated with a new list of start/continue characters.
  - Tom added that this is why we want to defer to the implementation-defined Unicode version.
  - Steve added this is also why we want the identifier stability guarantee; otherwise we get linkage problems.
  - Peter suggested it should be ok to define a profile with `<Start>` defined as `XID_Start + _`, and `<Continue>` defined as `XID_Continue - <all_combining_characters>`.
  - Steve noted that we have a floating Unicode reference today.
  - Tom agreed but noted that we have not yet required implementors to state which version of Unicode they conform to.
  - Steve agreed and added that, technically, we only have a floating reference to ISO/IEC 10646; this may not cover the normalization algorithm.
  - Steve summarized some options; there are two ways to deal with NFC: 1) source must be NFC normalized, and 2) the compiler internal encoding must be NFC. Not allowing combining characters gives us the stability that we need without having to distinguish between those options.
  - Steve continued that omitting combining characters avoids the problem of Unicode introducing new precomposed characters that previously had to be represented with a combining character thereby changing NFC.
  - Tom responded that he thought the Unicode standard has a stability guarantee that new precomposed characters will not be introduced.
  - Peter observed that allowing combining characters is therefore required for new characters.
  - Tom suggested we need to do some more research.
  - Steve, after checking the Unicode standard, reported that normalization forms are guaranteed to be stable.
  - Zach quoted from section 3 of [UAX #15](#):

- "It is crucial that Normalization Forms remain stable over time. That is, if a string that does not have any unassigned characters is normalized under one version of Unicode, it must remain normalized under all future versions of Unicode."
- Peter repeated his guidance that combining characters must be allowed in order to support some scripts.
- Tom agreed and acknowledged that we probably therefore need to require NFC.
- Tom summarized options identified so far:
  - 1) The compiler converts to NFC internally. This could technically break some existing code.
  - 2) Require UTF encoded source files to be NFC and that non-UTF encoded source files be transcoded (noting that we believe that transcoding from any existing character sets will produce NFC).
- Zach observed that the implementation effort is equivalent for those cases since an NFC check can bail out early if the check fails, but is otherwise same amount of work so that the complexity cost is the same.
- Steve stated that he may not be in Prague, but that others can champion the paper as needed.
- Peter and Zach both volunteered to champion.
- Steve stated he would try to get an updated revision submitted for the Prague pre-meeting mailing.
- Tom stated that the next meeting will be January 22nd.

## January 22nd, 2020

### Draft agenda:

- [P1885R0: Naming Text Encodings to Demystify Them](#)
  - Follow up on recent mailing list discussions.
  - Identify and discuss intended use cases.
- [P2020R0: Locales, Encodings and Unicode](#)
  - General discussion, corrections, suggestions, etc...

### Attendees:

- Corentin Jabot
- David Wendt
- Hubert Tong
- Jens Maurer
- Peter Bindels
- Steve Downey
- Tom Honermann
- Zach Laine

### Meeting summary:

- [P1885R1: Naming Text Encodings to Demystify Them](#)
  - Tom introduced the topic for discussion:
    - SG16 approved P1885R0 to forward to LEWG in Belfast.
      - <http://wiki.edg.com/bin/view/Wg21belfast/SG16P1885R0>
    - Corentin has now provided an R1 with minor updates.
    - Since then, concerns were raised on the SG16 mailing list:
      - <https://lists.isocpp.org/sg16/2019/12/0993.php>
      - (See email thread continuation in January as well)
    - Questions of use cases have been raised.
  - Corentin stated that use cases haven't changed from his perspective and that the discussion on the mailing list went off on a tangent.
  - Tom replied that the discussion suggested a lack of consensus on the importance of a name vs a MIB ID.
  - Corentin stated that what is proposed is just a name intended to resolve issues with names not being portable across platforms. The proposal relies on MIB IDs to correlate names for use with third party products. The proposal does not allow dynamically adding names so as to avoid the possibility of inconsistent results.
  - Tom asked what the motivation was for not including enumerators for all MIB IDs in `text_encoding::id`, but to require the implementation to support all names and aliases from the [IANA Character Set Registry](#).
  - Corentin replied that the requirements were changed in R1. Hosted implementations are now required to support all of the names, but freestanding implementations need not.
  - Tom asked for clarification regarding omission of enumerator IDs.
  - Corentin replied that, if we specify enumerator names for all registered character sets, then we'll have to maintain that list. Additionally, if implementors can add names, that could lead to portability or compatibility issues. Discussion with others prior to Belfast suggested more names were not needed.
  - Jens summarized the concern; the RFC has ~150 names and we would have to put all 150 names into the enumeration and deal with the maintenance. If we select just a few names, then we don't have a maintenance burden.

- Tom countered that use of the `cs` prefixed identifiers described in section 2.3 of [RFC 2978](#) and maintained in the [IANA Character Set Registry](#) would avoid the portability and compatibility concerns and provide a specification we can defer to.
- Corentin replied that it isn't quite that simple because of version skew and that exposing MIB IDs to programmers has limited value to begin with.
- Tom countered that, in the example use case provided in Belfast, you don't necessarily know what the name is.
- [ *Editor's note: That example use case is:*

```
template<class traits, class Rep, class Period>
void print_fancy_suffix(basic_ostream<char, traits>& os, const duration<Rep, Period>& d)
{
    if constexpr (text_encoding::literal().mib == UTF-8) {
        os << d.count() << "\u00B5s";
    } else {
        os << d.count() << "us";
    }
}
```

]

- Corentin replied that the use case could still be covered by comparing the implementation provided `text_encoding` object with one constructed by the programmer with a name.
- [ *Editor's note: Presumably something like:*

```
template<class traits, class Rep, class Period>
void print_fancy_suffix(basic_ostream<char, traits>& os, const duration<Rep, Period>& d)
{
    if constexpr (text_encoding::literal() == text_encoding("UTF-8")) {
        os << d.count() << "\u00B5s";
    } else {
        os << d.count() << "us";
    }
}
```

]

- Tom opined that string names are good for interaction with current third party libraries, but IDs are preferred for the example provided
- Corentin replied that adding more enumerators is ok, but expressed discomfort with deferring to the IANA registry due to the possibility of incompatibilities arising from version skew.
- Steve noted that the proposal only intends to provide portable names; there is no requirement for encoders and decoders to be provided.
- Zach observed that no enumerator is provided for Windows-1252 and asked how an implementor that frequently traffics in that encoding would provide support.
- Corentin responded that a `text_encoding` object can be constructed by name or that the fixed numeric value from the IANA registry can be used.
- JeanHeyd asked if we could reserve a range of MIB IDs for use by implementations similar to the Private Use Area in Unicode.
- Corentin replied that he is strongly opposed to doing so.
- Corentin asked if we really want all of these names to be available as identifiers when we can just use strings.
- Zach responded that he thinks it makes sense for cases where we know compilers default to certain encodings.
- Corentin repeated that he doesn't want implementors to add their own names.
- Jens asked about the source for the names whether as strings or identifiers. [RFC 3808](#) lists the MIB names with interesting spellings, and [RFC 2978](#) defines a registration process, but neither provides the latest names.
- Steve provided the URL to the IANA registry and explained that the RFCs don't change, but specify the URL for the registry; which doesn't change often.
  - <https://www.iana.org/assignments/character-sets/character-sets.xhtml>
- Tom added that the IANA registry mostly changes for administrative reasons, not because of new character set registrations.
- Jens asked how it is determined which names are good for enumerators.
- Tom replied that [RFC 2978](#) specifies that each registered character set have an associated name prefixed with "cs" that is appropriate for use as an identifier.
- Jens asked why the names in the proposal do not match the "cs" names.
- Corentin responded that he picked names that he preferred.
- Jens asserted that, in that case, implementors cannot extend the list.
- Zach stated that there isn't much cost in taking the list of "cs" prefixed names, removing dashes, and dumping that list in the wording and asked again for motivation for omitting them.
- Corentin replied that he thought they were not needed.
- Zach agreed that many would not be used much, but determining which ones are important would be difficult where as just including them all would be easy.
- Tom asked Corentin, why he felt comfortable deferring to the IANA registry for string names, but not for enumerator names
- Corentin replied that he felt that the names and alias names were definitive, but that the enumerator names seemed more fuzzy.
- Corentin asked Jens if there are concerns regarding the use of trademark names in the standard; many of the character set names include trademark names.

- Jens replied that we already use trademarked names like Windows and POSIX in the filesystem specification.
  - Steve added that these names have already been vetted by their respective owners, if necessary, for inclusion in the registry.
  - Jens asked if the names in the IANA registry might already be reflected in an ISO standard that we could reference instead.
  - Corentin replied that he was unaware of such an ISO standard.
  - Tom asked Jens how a search for such an ISO standard could be conducted.
  - Jens suggested searching for "character set" in the ISO list.
  - Steve noted that the RFC describing the IANA registration process does mention ISO standards such as ISO 10646, ISO 8859, and ISO 2022.
  - Corentin stated that web browsers, iconv, ICU, etc... all use the IANA registry; it is the defacto standard.
  - Jens expressed some uncertainty with regard to how to refer to these RFCs from the standard, but mentioned that we did similarly for the time zone database which is even less regulated.
  - Jens raised a concern about impact to small/embedded implementations. As proposed, they would have to include an instance of the string name table with every instance of the program and that could be problematic even for some hosted implementations.
  - Tom suggested that, if the string table is not referenced; e.g., if none of the `text_encoding` factory functions is referenced or if the `<text_encoding>` header is not included, that the implementation might be able to omit it.
  - Jens suggested that it would be helpful if the paper addressed cost of implementation and anticipated impact to deployments.
  - JeanHeyd suggested that the guarantee we make should be that if only `text_encoding::system()` OR `text_encoding::literal()` are called, then there should be no string table overhead.
  - Jens asked if an implementation could provide support for a reduced set of names. If not, the discussion of how to reduce deployment cost is warranted since, as proposed, this is not a zero-cost of zero-overhead solution.
  - Jens also stated a preference for the `system()` and `wide_system()` functions to return a MIB ID rather than a `text_encoding` object.
  - Corentin responded that there may be cases where the system encoding is not registered with IANA. In that case, the MIB ID would be "unknown"; and a different interface would have to be used to retrieve the string name of the encoding anyway.
  - JeanHeyd provided WTF-8 and Modified UTF-8 as examples of encodings that are not registered with IANA but that are known to be in use on Android and elsewhere on the web.
  - Jens suggested that, in such cases, the implementation register their encoding.
  - Zach asked to clarify what the motivation is for supporting string names at all.
  - Tom responded that third party products like iconv and ICU have interfaces that require use of string names.
  - Corentin confirmed.
  - Tom added that the IANA registry is effectively a common subset of recognized names.
  - Zach stated a preference for omitting string names and just relying on MIB IDs.
  - Corentin responded that doing so would complicate use of iconv.
  - Hubert expressed a lack of motivation for an interface that relies on numeric values that no one knows; the string names make sense.
  - Jens pondered if string name to MIB ID lookup was an orthogonal feature.
  - Tom stated that question was posed in the mailing list discussion as well.
  - Corentin mentioned existing host system interfaces. Windows provides a code page with an ID. POSIX systems provide a name and no ID.
  - Jens suggested that an interface that provides a string name does not suit all use cases. For example, a programmer might desire to assert a specific system encoding; that shouldn't require a full string table.
  - Zach expressed a desire for the interface to provide more safety and that he would prefer a list of identifiers over a list of string names.
  - Hubert suggested other benefits of the string names, 1) useful for interaction with the system and third party libraries, and 2) useful for interchange or serialization.
  - Hubert expressed concern about use of a string interface for looking up an encoding name and asked what name is provided in response to a lookup of a MIB ID.
  - Corentin replied that there is no proposed lookup interface that accepts a MIB ID. The factory interfaces like `text_encoding::system()` return a preferred name, but otherwise, the name provided when constructing a `text_encoding` object is preserved.
  - Jens expressed a desire for a low-level interface that just returns an integer that could be used to assert the environment is UTF-8 without having to compare with a bunch of strings; that could be a zero overhead facility.
  - Hubert asked if there is overhead if neither of `text_encoding::system()` OR `text_encoding::wide_system()` is called.
  - Corentin responded that yes, there is, but it is low.
  - Hubert cautioned that some standard library implementors are likely to oppose anything that increases startup cost or requires "static constructors".
  - Tom asked why the interface couldn't perform a lazy lookup.
  - Corentin responded that calls to `setlocale()` could interfere; `text_encoding::system()` is intended to return the locale dependent encoding known at program startup time.
  - [ Editor's note: Later discussion on the SG16 mailing list revealed that it is possible on POSIX systems to retrieve the locale dependent encoding known at program startup time regardless of intervening calls to `setlocale()` with code like:
- ```

    locale_t loc = newlocale(LC_CTYPE_MASK, "", (locale_t)0);
    const char* name = nl_langinfo_I(CODESET, loc);
    ...
    freelocale(loc);
]

```
- Hubert suggested that programmers can collect this information on their own and that they should be aware if some library is calling `setlocale()` before `main()` is invoked.

- Tom agreed, but stated that doing so is hard in practice, particularly for library authors.
- JeanHeyd observed that the C library behavior depends on the currently set locale and asked what benefit is provided by `text_encoding::system()` if it's not in sync with the C and C++ libraries.
- Tom responded that it indicates what encoding is expected for I/O outside of the process.
- Tom confirmed that the next meeting will be on February 5th and that it will be the last meeting before we meet in Prague.

## February 5th, 2020

### Draft agenda:

- Preparations for Prague.
- [P2020R0: Locales, Encodings and Unicode](#)
  - General discussion, corrections, suggestions, etc...
- [P1629R0: Standard Text Encoding](#)
  - Status update from JeanHeyd.

### Attendees:

- Amanda Kornoushenko
- Corentin Jabot
- JeanHeyd Meneide
- Jens Maurer
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann

### Meeting summary:

- Preparations for Prague.
  - PeterBr asked if it would be possible to attend the SG16 meeting in Prague remotely.
  - Jens provided some background:
    - Historically, remote attendance has not been allowed for a variety of reasons including but not limited to:
      - voting concerns
      - confidentiality concerns
      - attendance by people not familiar with our processes
    - SG and WG chairs have facilitated remote attendance on occasion for paper authors or well known experts.
    - For Prague, the Corona virus outbreak has prompted an exception for this meeting for paper authors.
    - SG chairs can choose to facilitate remote attendance subject to technology support.
  - PeterBr stated that the UK national body is considering making a request to enable remote access for people facing attendance challenges due to issues like VISA access and child care responsibilities.
  - Tom indicated that a member of the UK national body had reached out to him to ask how SG16 conducts our telecons.
  - Jens stated that the number of meeting attendees is raising logistical and hosting challenges.
  - Jens added that hotel wifi may not suffice for video conferencing.
  - Tom asked if Jens could provide audio gear that SG16 could use to facilitate remote attendance.
  - Jens confirmed he could.
  - Tom confirmed that a best effort approach will be made to facilitate remote attendance via BlueJeans.
  - Tom asked everyone to review the tentative schedule
    - <http://wiki.edg.com/bin/view/Wg21prague/SG16>
  - Jens suggested removing the Wednesday afternoon time slots since we don't have a room reserved for that time.
  - Tom agreed to do so. [ *Editor's note: and did so.* ]
- Tom announced that the newly formed Message Formatting Unicode Working Group has been invited to attend the SG16 meeting planned for March 11th.
  - <https://github.com/unicode-org/message-format-wg>
- [P2020R0: Locales, Encodings and Unicode](#)
  - Corentin introduces:
    - The paper should be updated to include additional motivation for support of localization in the standard library. However, since locale support is already present in the standard library, there is a desire not to distract from other topics and attempt to motivate unnecessarily.
    - The intent of the paper is to provide guidance and establish direction.
    - The existing locale support in the standard library is deficient, but in use regardless. We could seek to deprecate it in favor of new facilities, though actually marking these interfaces `[[deprecated]]` could be problematic for some projects due to compiler warnings.



- A primary point in this paper is that character encodings and locales are distinct concerns, though they have been historically conflated, particularly in POSIX.
- High quality locale support depends on Unicode algorithms.
- Attempting to provide locale support for non-UTF encodings is not realistic.
- Whether a character is an uppercase or lowercase letter is not locale dependent. However, case conversion is locale dependent.
- The existing character classification functions are deficient since they operate on code units as opposed to sequences of code units or code points.
- New locale facilities must be Unicode based. For support of legacy encodings, conversion to Unicode and back will suffice.
- Iostreams and locale are closely tied and operate on individual code units. Proper localization cannot be provided using code unit based operations.
- Linux and macOS deployments nearly exclusively use a UTF-8 locale. At program startup, the program locale is set to "C". Setting it to "C" is desirable; programs should opt-in to localization behavior. However, setting it to "C" also has the effect of changing the encoding and that is not desirable.
- PeterBr stated that support for multiple encodings is not required for locale support.
- Corentin agreed with a caveat; that is true for Unicode encodings, but when not using Unicode encodings, switching locales requires switching encodings.
- Steve stated that, for one of their internal facilities, encoding is important for understanding what is provided by the locale library since the library provides char based interfaces and the localization data is encoding dependent. It would be possible to retranslate all existing messages, but would be a large undertaking and getting localization wrong is expensive. We need to enable bridges to the past.
- PeterBr noted that we can't expect UTF-8 locale support on Windows; it will be UTF-16.
- Corentin responded stating that is ok so long as it is Unicode since conversion to UTF-8 doesn't lose data.
- Steve remarked that it is impressive how much locale related code works by accident.
- Corentin supplied a list of operations affected by locale: case mapping, collation, search. Search was surprising; it depends on locale because matching base characters is desired for some locales, but not for others. The break algorithms are also locale dependent.
- Corentin added that, thanks to Han unification, text rendering is locale dependent.
- Jens observed that section 4 omits some items; number formatting for example.
- PeterBr stated it would be helpful to have some Japanese contributors in SG16 to answer questions about formatting in their locales.
- Corentin opined that it is best to just follow Unicode; it specifies how to handle locales.
- Corentin added that, in some locales, multiple numeric formats may be used. For example, in India.
- PeterBr asked if the character classification functions could be deprecated if replacements were made available.
- Corentin replied that they tend to be used in cases where programmers explicitly expect ASCII.
- Jens stated that we all likely agree that the current character classification functions are deficient and that new facilities are required in order to deprecate them.
- PeterBr asked how likely implementors are to be willing to ship a Unicode DB.
- Corentin suggested that discussion of that be postponed to discussion of [P1628 - Unicode character properties](#).
- Jens objected to the idea of defaulting the program's startup encoding to UTF-8 if the environment specifies a UTF-8 encoding. The "C" locale only supports the basic execution character set, e.g., ASCII. Programs that want to support extended characters should call `setlocale(..., "")` at program startup to opt-in.
- Corentin stated that the choice C made to default the locale to "C" was a good choice for formatting facilities; many programs aren't intended to produce locale dependent formatting. But encoding is different.
- Steve stated that the as-if rule is leaned on here as implementations don't actually call `setlocale(..., "c")` during startup; adding a call to `setlocale(..., "")` would be a major change.
- Tom clarified that Corentin's intent is only to adopt the encoding from the environment locale on program startup, not the locale formatting settings. The proposal states that if, for example, the locale specified an encoding of UTF-8, then the as-if call on program startup would be something like `setlocale(LC_ALL, "C.UTF-8")`.
- Corentin acknowledged that the way C and C++ are tied is a valid concern and that implementors depend on the underlying OS for locale support.
- Jens expressed skepticism regarding the ability to change the program startup behavior, suggested that a new interface to retrieve the environment specified encoding be provided, and that `setlocale` be left alone.
- Tom agreed that new functionality doesn't have to follow current behaviors.
- Jens continued; new functionality can be provided that is independent of `setlocale`.
- Jens opined that "C.UTF-8" doesn't make sense.
- Tom stated that Python made a change to assume UTF-8 for the "C" locale and that the Python [PEP-538: Coercing the legacy C locale to a UTF-8 based locale](#) and [PEP-540: Add a new UTF-8 Mode](#) documents are informative for motivation.
- PeterBr asked if use of the "C" locale implies use of the encoding of the execution character set.
- Jens replied that, effectively, yes, but wording could make that more clear.
- Tom stated his goal for this paper in Prague; to poll a subset of the possible future directions listed in section 6 to establish priorities for them; we can then identify tasks and papers to write.
- Jens expressed a desire for a road map for the future before moving forward with deprecation.
- PeterBr agreed and stated that a desire for a road map underscored the need for good motivation.
- Corentin stated that we need to identify all of the current implicit and explicit locale dependencies.
- PeterBr suggested incremental improvements may be made by adding overloads with explicit locale parameters.
- Tom suggested that we spend an evening in Prague going through the paper in more detail with the intent to improve presentation and fill in gaps.



- JeanHeyd introduced:
  - [N2440](#) and [P1629](#) are intended to provide text decoding, encoding, and transcoding interfaces.
  - These replace `wstring_convert` and friends.
  - UTF-8 is difficult to enforce in `char` because non-UTF-8 data arrives in `char` based strings.
  - C provides few useful interfaces in this area.
  - WG14 approved parts of N2440 and an implementation is available in a standalone library.
  - Plans to submit implementations of N2440 to glibc and musl libc are pending.
  - P1629 provides extensible encoding objects.
  - There is an implementation of P1629 that provides encoding, decoding, transcoding, validation, and code point counting services.
- Corentin asked what the motivation is for contributing new interfaces to C.
- JeanHeyd responded that it makes sense to do so and that WG14 has already approved direction to add the mbs to UTF conversion variants.
- Tom expressed his motivation for contributing to C; doing so reduces friction between the languages.
- PeterBr asked if the implementation of N2440 uses SIMD instructions.
- JeanHeyd replied that it does not yet, partially because the musl maintainers will want straight C implementations.
- Jens asked why the single-character interfaces are not proposed.
- JeanHeyd replied that only the restartable variants are being proposed.
- Jens stated that WG14 will consider a WG21 approved proposal to qualify as implementation experience.
- PeterBr asked about `replacement_code_unit` and how a replacement character that requires multiple code units would be specified; it seems like the replacement character should be provided as a string.
- Tom expressed confusion about the existence of `replacement_code_unit` as he expected a replacement code point to be specified.
- [ *Editor's note: Tom later started an email thread on the SG16 mailing list regarding this: <https://lists.isocpp.org/sg16/2020/02/1101.php>.* ]
- JeanHeyd replied that a replacement code point is preferred and that the replacement code unit is a fall back.
- Corentin stated that he didn't think `replacement_code_unit` is needed at all.
- JeanHeyd replied that it is used to distinguish between errors happening in different encode/decode directions.
- Steve suggested that, perhaps, better names are needed to communicate their intent.
- JeanHeyd replied that he will update the replacement names and make them ranges or strings.
- Jens observed that the design appears to be all compile-time based and asked how run-time dependent encoding is handled.
- JeanHeyd replied that the compile-time implementation can be wrapped in a run-time design.
- Jens expressed a desire to see the design specified in terms of concepts.
- JeanHeyd replied that the proposal will use concepts, but that the current implementation is intended to work with pre-C++20 compilers.
- Tom asked how much of the proposal is implemented.
- JeanHeyd replied that interfaces have been implemented for decoding, encoding, transcoding, validation, and code point counting.
- JeanHeyd added that support for normalization hasn't been completed, but normalization can be done later.
- Tom stated that the next meeting will be February 26th and the focus will be on post-Prague follow up.

## February 16th, 2020

### Draft agenda:

- Post-Prague follow up.

### Attendees:

- Corentin Jabot
- David Wendt
- JeanHeyd Meneide
- Jens Maurer
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Zach Laine

### Meeting summary:

- [P2071R0: Named universal character escapes](#):
  - Tom provided a status update. Waiting on a response to add an additional co-author. Working on updates to address EWG feedback. Wording will need to be re-done for P2029. On track for review by EWG again in Varna; will hopefully be made tentatively ready then.
- [P2029R0: Proposed resolution for core issues 411, 1656, and 2333; numeric and universal character escapes in character and string literals](#):
  - Tom stated an update will be submitted for the Prague post-meeting mailing with the intent that it be discussed at the next core issues processing meeting.

- Jens stated that the next core issues processing meetings are planned for March 23rd and April 20th.
- Renaming *universal-character-name*:
  - Corentin brought up an [email](#) that he had sent to the SG16 and core mailing lists regarding a desire to rename *universal-character-name* to *universal-character-codepoint* since no names are actually used in these productions (code point values are).
  - Jens stated that it can be difficult to get consensus on a change via the core reflector; such a change needs core buy in.
  - Corentin asked how he should proceed.
  - Jens replied that the issue could be discussed in the next core issues processing meeting. The process to get an item on the agenda for those meetings is to get it on the CWG wiki page for Varna, but the Varna wiki hasn't been populated yet. Jens said that he would poke at someone to get the wiki structure in place.
  - Tom asked for more details about this process and whether it is really ok to get changes like this initiated without a paper or core issue.
  - Jens replied that a paper is best to ensure proper attention and progress.
  - Jens added that an updated core issues list hasn't been published for some time now.
  - [ *Editor's note: The last published core issues list is revision 100 and has a date of 2018-04-11.*  ]
  - Jens asked if *universal-character-codepoint* is what we want and suggested *unicode-code-point* as an alternative.
  - Tom expressed support for Jen's suggested alternative.
  - Zach expressed a preference for something more specific since this production is for one particular way to express a code point.
  - Jens responded that this is a grammar term and asked if the grammar term for P2071 should also be named *something-codepoint* because it designates one.
  - Corentin expressed support for that sentiment.
  - Zach asked if the implication is that both `\UNNNN` and `\N{...}` would fall under *unicode-code-point*.
  - Jens replied that they are distinct because `\UNNNN` can be generated from characters not in the basic source character set that appear in identifiers in the source code. `\N{...}` likely gets effectively quickly translated to `\UNNNN`.
  - Jens opined that the name isn't super important since this is just a grammar term and people should expect to have to look up exactly what it means.
  - Zach stated that *unicode-code-point* seems like the right choice then.
  - Tom asked about using the names *unicode-code-point* for `\UNNNN` and *unicode-code-point-name* for `\N{...}`.
  - Corentin suggested that, in P2071, *named-escape-sequence* be renamed to *unicode-named-escape-sequence* for the `\N{...}` case.
  - Jens asked if `\N{...}` is allowed in identifiers.
  - Tom replied that it is not.
  - Jens noted that being a significant difference from `\UNNNN`.
  - Tom expressed some hesitation with regard to adding "unicode" to *named-escape-sequence* since, in theory, we could support non-Unicode names like D does.
  - [ *Editor's note: D uses HTML 5 entity names for its named character escapes.*  ]
  - Jens expressed a preference for *named-escape-sequence* as it is simple and matches nearby grammar terms like *octal-escape-sequence* and *hexadecimal-escape-sequence*.
  - PBindels asked about just using *code-point* for `\UNNNN`.
  - Corentin stated that Unicode is needed in this case.
  - Jens agreed noting that the syntax is specific to Unicode code points.
  - Jens asked to confirm that there is no requirement for an implementation to have a list of acceptable or unacceptable code points for `\UNNNN` other than for surrogate code points and the range of code point values (0-0x1FFFF).
  - Tom confirmed; implementations are not required or allowed to map an unrecognized code point to a replacement code point.
  - Jens acknowledged and added that programs that specify an unassigned code point will not be rejected either.
  - Jens asked if naming this *unicode-code-point* implies a valid character.
  - Steve suggested that, perhaps, the right name is *unicode-scalar-value*.
  - Everyone expressed profound distaste for the scalar value term.
  - Jens suggested that P2071 be updated to add editorial direction to rename *universal-character-name* to *unicode-code-point*.
  - Tom agreed to do so.
  - [ *Editor's note: concurrent with this meeting, a prominent core member replied to Corentin's email and requested that we retain the universal-character-name name since it has been in use in C99 and C++23 for 20 years now and is referenced in existing literature. Due to there being opposition to the name change, Tom then decided not to pursue the editorial rename via P2071. Anyone wishing to pursue the rename should therefore write a separate paper.*  ]
- [P0592R4: To boldly suggest an overall plan for C++23](#):
  - Tom introduced the topic. At plenary in Prague, Peter Bindels asked what the process would be to amend P0592. Tom was approached by several committee members arguing that Unicode support should be added as a priority for C++23. Tom is concerned about spending committee time addressing a problem that might not exist and is worried that attempting to add our favorite topic to the priority list might inspire other groups to argue for adding theirs potentially consuming significant amounts of committee time.
  - Jens asked what papers aren't making progress.
  - Tom replied that, right now, SG16 is the bottleneck for SG16 work. The EWG and LEWG chairs have been quite supportive of making progress on Unicode matters.
  - PBindels stated that he raised this in plenary partially to encourage people to write papers; we want to ensure that EWG and LEWG are prepared for additional work that builds on the ground work we've been laying.
  - Steve suggested a potential bad scenario. Two years from now, there could be a glut of pattern matching papers consuming lots of committee time just as C++23 is wrapping up.
  - Tom agreed that is a possible concern, but added that he doesn't think we can preempt it.

- PBindels stated that P0592 is meant as a general guideline and that we need a plan for ourselves so that we know what we are aiming for in C++23.
- PBrett agreed and added that knowing what we want, when we want it by, and who is responsible would be helpful.
- Tom responded that the SG16 github site has issues tracking the SG16 work currently in motion as well as other tentative ideas. Many of those issues are marked as "help wanted" and are available for volunteers to take on.
- Tom added that [P1238](#) is due for an update. That paper could be updated to list items that we want help with. Perhaps we need to do more outreach to enlist additional help. We could post help wanted tweets or ask more people to get involved when giving talks.
- [P1949R1: C++ Identifier Syntax using Unicode Standard Annex 31:](#)
  - Tom summarized the current status; Steve has been preparing a revision following EWG review in Prague.
  - Steve stated that the paper is ready for the post-meeting mailing.
  - Tom stated that we lost the tentatively ready status in EWG for this paper following [additional email discussion](#) that raised concerns about possible undefined behavior in conjunction with token pasting.
  - Tom asked if the paper needs to address token syntax as well as identifier syntax.
  - Corentin replied that NFC checking should happen after preprocessing.
  - Tom asked if the grammar for identifiers is relevant for tokens.
  - Jens replied that *preprocessing-token* is distinct and that they get converted into identifiers, keywords, etc... at a particular translation phase.
  - Jens added that this occurs in translation phase 7 per [\[lex.phases\]p1.7](#) and [\[lex.token\]p1](#). Core language wording should be added here to state that an identifier shall be in NFC form.
  - [ *Editor's note: In the draft wording, this is added to [\[lex.name\]](#). ]*
  - Steve asked if this is a "shall" or "must" situation.
  - Jens replied that "shall" is correct because a diagnostic is required and that "must" is a forbidden term in normative wording.
  - Tom mentioned that Peter Bindels has drafted a paper arguing for P1949 to be designated as a DR for C++20.
  - Jens stated that the DR process would be to get P1949 adopted for C++23, and then get a plenary straw poll to apply it as a DR against C++20.
  - Steve agreed to add content to the paper arguing for treating the matter as a DR.
  - PBindels told Steve to take whatever content he wanted from his DR draft and that he will abandon it.
    - [Peter's draft](#)
  - Jens suggested putting content directly in both the paper's front matter and at the beginning of the core wording that this be considered a DR.
  - Jens added that to ensure this is highlighted in core discussion.
  - Tom asked if there would then be two core motions. One to accept the paper for C++23 and another to accept it as a DR for C++20.
  - Jens replied that it could be one motion: "adopt for C++23 and consider as a DR for C++20"
- [P1844R1: Enhancement of regex:](#)
  - Tom summarized the Prague outcome. We declined to forward this paper on and Peter Bindels has authored a draft paper to deprecate `std::regex`.
  - Tom apologized for not yet reviewing Peter's deprecation paper.
  - PBindels stated he is waiting for review feedback from a select group of reviewers before sharing the draft more widely.
  - PBindels summarized the paper; it includes rationale for why programmers shouldn't use `std::regex`, performance numbers, ABI issues, votes in Prague, etc...
  - PBindels added that he would like to add more details on why the Visual C++ implementation is presumably so heavily impacted by ABI concerns.
  - Tom replied that the Visual C++ implementation exposes the entire state machine in template instantiations, so no changes can be made that affect the state machine.
  - Corentin suggested additional content stating that the design is overly complicated because it supports so many regex languages and that the requirement to do so impacts performance.
- [P1885R1: Naming Text Encodings to Demystify Them:](#)
  - Tom suggested adding a list of encodings that are supported by ICU, iconv, Windows, etc..., but that are not present in the IANA database.
  - Corentin replied that he would try to do so.
  - Tom suggested that we try to register an encoding to see how burdensome doing so is to assuage fears about support for unrepresented encodings.
  - Corentin replied that he didn't think that would be a good use of our time.
  - PBrett added that doing so would be an abuse of process; the IANA registration process should only be used to register encodings for which there is a demonstrable need.
  - Tom acknowledged and agreed.
  - Tom suggested adding additional use cases to the paper to make it more evident to LEWG(I) how this functionality is expected to be used.
  - Tom added that such use cases should include intended future direction as well; e.g., interaction with [P1629: Standard Text Encoding](#).
  - Corentin agreed to do so.
  - JeanHeyd stated that he would look into prototyping integration between P1885 and P1629 and making it available on godbolt.
- [P1953R0: Unicode Identifiers And Reflection:](#)
  - Tom expressed uncertainty as to what the next steps are.
  - Corentin stated that we need to get P1949 accepted first. We can then revisit reflection and provide our recommendations to SG7.
  - Corentin added that we do need to keep on top of what SG7 is targeting for C++23.

- [P1040R5: std::embed](#):
  - Tom summarized the Prague outcome. EWG expressed support for a file/directory handle #depend based solution. That solves SG16 concerns; unless there is still a need to support paths that are relative to a handle at translation phase 7.
  - JeanHeyd stated that there are lots of issues with the VFS/node/handle approach. But regardless, the SG16 recommendation to use `char8_t` and UTF-8 resolves any SG16 related concerns.
- Tom stated that the next meeting will be March 11th and that we'll be meeting with the new Unicode Message Format Working Group.

## March 11th, 2020

### Draft agenda:

- Meet and greet for SG16 and the new Unicode Message Format Working Group (MFWG)
  - Individual introductions.
  - A brief history of each group.
  - Current efforts and plans for each group.
  - General discussion of message formatting.
  - Discussion of how we might work together to mutual benefit.

### Attendees:

- Amanda Kornoushenko
- Corentin Jabot
- Elango Cheran
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Markus Scherer
- Mihai Nita
- Peter Brett
- Romulo Cintra
- Shane Carr
- Steve Downey
- Steven R. Loomis
- Tom Honermann

### Meeting summary:

- The meeting started off with a round of introductions.
- Tom provided a brief history of SG16 and changes championed for C++20.
  - Tom mentioned the work that went into `std::format` via [P1868](#) in order to produce correctly aligned text for monospaced presentation formats.
  - SLoomis stated that character display width is an important problem that is deserving of its own project.
  - PBrett asked if there were plans to enable `std::format` to handle text translation.
  - Tom stated that our current direction is captured in [P1238: SG16 Unicode Direction](#).
  - Markus provided some references for work being done in ICU to address C++20 incompatibilities:
    - <https://github.com/unicode-org/icu/pull/979> (a pull request providing minimal changes to allow ICU to compile with C++20; basically a bunch of added `reinterpret_cast` casts for uses of `u8` string literals to continue using them as arrays of `const char`)
    - <https://unicode-org.atlassian.net/browse/ICU-20984> (a proposal for a more principled change that avoids the need for many of the `reinterpret_cast` casts)
- Members of the MFWG provided an introduction and current status summary.
  - Romulo gave a general introduction:
    - The initial impetus for the group was the observed demand for client side message formatting and a lack of browser features needed to effectively enable it two years ago.
    - There are currently a number of libraries available that cumulatively account for millions of weekly downloads:
      - NPM trends: <https://www.npmtrends.com/i18n-vs-i18next-vs-messageformat-vs-polyglot-vs-intl-messageformat-vs-fluent-vs-fbt-vs-format-message>
      - Overview and analysis of various libraries: [https://docs.google.com/presentation/d/1RujNFCq3gH9TUEKDB\\_uFdKWNG1A1j2\\_NBCdnTmnEqv0/edit#slide=id.g4af2a8f783\\_1](https://docs.google.com/presentation/d/1RujNFCq3gH9TUEKDB_uFdKWNG1A1j2_NBCdnTmnEqv0/edit#slide=id.g4af2a8f783_1)
    - A recommendation was provided to join [ECMA TC39](#) and contribute to the group chaired by Shane Carr that is responsible for [ECMA-402](#).
    - Discussed the idea of a new group with Shane focused on message formatting last year.
    - Shane brought lots of new people, got them talking, and worked with the Unicode consortium to create the new group.

- Shane continued:
  - Message formatting was already recognized as an item to focus on for ECMAScript.
  - The problem isn't unique to ECMAScript; it is a big problem space.
  - A big question is, how much of the localization stack is to be covered?
  - The Unicode consortium formed the new group in January, 2020 as a Unicode subcommittee.
  - Romulo was named chair of the new group.
- Mihai continued with an overview of the scope and design direction:
  - Think of message formatting like a locale aware implementation of `printf`.
  - But one that can handle plurals.
  - The idea is to separate the string from the localization data model.
  - For `printf`, the format is a sequence of parts, each of which contributes raw text or a placeholder with formatting data.
  - Proper internationalization requires a more complicated data model.
  - There are three major pieces needed:
    - The data model.
    - A serialization form.
    - A message store.
  - A goal is to provide a standard data model that can be mapped to various localization interchange formats; not to produce yet another message format.
- Romulo wrapped up:
  - Have had 5 meetings so far.
  - Are still in the design and requirements discovery phase.
  - Are still working on design processes to ensure efficient operation.
- General discussion ensued:
  - PBrett asked about challenges faced where ICU is currently deficient.
  - Mihai responded that he has a document he can share:
    - A major challenge is that ICU has the only widely deployed formatting implementation.
    - There are some ECMAScript libraries.
    - ICU does not support inflections well. For example, in English, one might say "the book", but in other languages, instead of inserting "the", the word "book" is changed.
    - ICU also doesn't handle combinations of plurals well. For example, a statement like "I bought 5 books and 2 posters" requires a complex nested switch format due to combinatorial explosion and the syntax is clunky.
  - Markus provided an example of plural complexities. Arabic has six different plural forms and more may be added to say "exactly one" or for "none". Two instances of pluralization in a message can lead to dozens of possibilities.
  - Shane stated that the ICU message format is the defacto standard right now, but there is no specification for it.
  - Elango responded that there are different defacto standards across different language ecosystems. Data literals in ECMAScript provide flexibility. Many programmers roll their own solutions and this results in inconsistency. The goal is to create a specification to encourage normalizing behavior across disparate implementations.
  - Jens commented on the complication of language bindings and extensive behavioral options.
  - Mihai agreed; message formatters congregate lots of functionality.
  - Mihai mentioned his prototype of a formatter that uses [Protocol Buffers](#) to translate syntax between different formatters. A comprehensive core data model is essential to be able to do so.
  - Elango stated that the general facility has just one output for message formatting; date formatting is provided by a plugin.
  - Mihai opined that it would be useful to have support for ranges as well.
  - PBrett stated that the ability to stream output is important for C++ and that this covers several orthogonal areas of concern:
    - The data model.
    - The abstract representation.
    - The concrete representation.
    - The sinks that messages go to.
    - Authoring of the translation database.
  - PBrett added that the above raises an important question for the MFWG to address: what are you trying to solve and what is the abstraction?
  - Jens observed that there appears to be little overlap between SG16 and the MFWG; when the MFWG specification is complete, SG16 will consume it.
  - Tom provided some of his perceptions of the benefits of working together. First, we ensure that the output of the MFWG works for purposes that we envision. Second, we get informed about infrastructure requirements that may require new facilities to meet in order to adopt the MFWG output. For example, enhancements to locale support.
  - Jens noted that, for `std::format` to be able to provide message formatting, it would have to be able to access the message catalog.
  - PBrett observed that adding that complexity to `std::format` may be challenging; it may be difficult to separate dependencies.
  - Markus noted that the mechanism used to pluralize a message is distinct from the source of the pluralization data.
  - Tom asked for clarification; pluralization is more of an algorithm than a lookup?
  - Markus responded that the way ICU has worked for the last 25 years is to parcel out strings or states, and then combine them according to specific rules.
  - Jens stated that WG21 has shied away from localization issues because the C++ story is so poor; serious work is needed here.
  - Mihai explained that, in the data model the MFWG is working on, a place holder is a cross reference to another string. If the mapping is generic, then a generic binding can be used, but loading can be customized.

- Jens expressed caution; we're wary of costs for features that are not used. Paying such costs is fine when needed, but should not pose overhead when not being used.
- Tom asked if the design can avoid such costs when such features aren't used.
- Jens responded that that isn't a fair question for a data model. A more appropriate question would be what the impact is to the tables used for pluralization data.
- Mihai responded that pluralization is not a large data set. The question is more relevant for inflection. Some languages are regular, but others are quite unregular and sorting requires a lot of data.
- Mihai added that message formatting brings algorithms together, but the information needed to guide the formatter is distinct.
- Jens asked if support for pluralization and inflections is in scope.
- Elango responded that pluralization is, but that inflections may not be; this is work in progress.
- Shane further responded that support for inflections is in scope as part of the effort to make a standard interface that enables plugging in extensions.
- Shane added that the focus is to provide a solution that does not require an implementor to implement everything. Implementors should be able to provide only the subset of features needed for a particular deployment.
- Markus elaborated on ICU's pluralization support. ICU doesn't attempt to determine the plural form for any word in any language. Rather, it identifies ranges of numbers for 100+ languages where things must be done differently. Translators are then required to author different messages. Translation tools are designed to prompt translators for each translation form that is needed. The pluralization form is then used for message selection.
- Jens observed that the ICU design is exactly what causes the combinatorial explosion.
- Mihai noted that the goal is to simplify the syntax, not to reduce the number of translations required.
- PBrett asked how translators can provide translations conveniently in those cases.
- Mihai responded that translation tools can do fuzzy matches and offer suggestions.
- PBrett asked about the MFWG road map; is it a goal to produce a Unicode TS?
- SLoomis responded, yes.
- PBrett asked if there is a tentative time line.
- Romulo responded that they are still in the design phase, so there is no clear road map right now.
- Mihai added that they have been focused on collecting use cases and feature requests; the rate of additions is decreasing.
- Mihai elaborated that getting consensus on a data model being a goal took some time.
- Corentin stated that it would be great to support a common syntax in C++ and ECMAScript so that translators have consistent experiences. For `std::format`, Python's syntax was adopted thereby enabling developers to switch easily.
- Mihai responded that a common format is anticipated for translators, but that does not necessarily correspond to what a programmer writes.
- PBrett hypothesized that a C++ implementation could have a `constexpr` solution that uses C++, but that can be translated to some other syntax.
- PBrett added that we need to think of how to deprecate and replace `std::message`.
- Jens noted that understanding the data model is important to envision how the various parts fit together and asked if a draft data model exists.
- Mihai responded that there are currently two documents on the data model. Elango provided a document that argues for a data model, and Mihai provided one that maps a model to one of several implementations and discusses how it can be modified to add features. There is no final draft.
  - Elango's doc: [https://docs.google.com/presentation/d/1fBfawWNfniCFox-PltCMYVtbcVwYGYkk78GUbWzas5o/edit#slide=id.g8254abe56c\\_0\\_0](https://docs.google.com/presentation/d/1fBfawWNfniCFox-PltCMYVtbcVwYGYkk78GUbWzas5o/edit#slide=id.g8254abe56c_0_0)
  - Mihai's doc: <https://docs.google.com/presentation/d/1dyW29SlqjPRZVScobqEXjnP29fhnbgMkCfgrPOWj3Tnw> (currently requires permission to access)
- Corentin asked if there is a reference syntax available and noted that none of us our linguistics experts.
- Mihai responded that a syntax for ECMAScript is anticipated along with a language independent form in a structured data format like JSON.
- Markus suggested that SG16 should consider whether it wants to do something in this area or whether this functionality should be left to non-standard libraries.
- Tom responded that the design direction would allow for a standard implementation, but doesn't restrict the use of non-standard implementations.
- Jens suggested that it would be good to appoint someone from SG16 to be a liaison to attend MFWG meetings and keep tabs on things.
- Shane agreed with that approach; it would help to maximize utility to consumers.
- Jens commented that what the committee needs is a standard that we can defer to that was produced by experts since, if left to our own devices, we'd probably produce a poor design. Dependence on such a standard needs to be figured into our road map.
- PBrett asked about the [MFWG mailing list](#) and [MFWG github site](#); neither looks particularly active.
- SLoomis replied that most activity is happening in github issues.
- Shane added that there is a [Slack Unicode channel](#).
- PBrett suggested that SG16 should discuss more, reflect, and contemplate how we want to move forward.
- PBrett asked what SG16 can do to help facilitate work by the MFWG.
- Mihai responded; try not to invent something new.
- Corentin noted that we need to re-design our locale facilities before we can take on message localization and that we need to implement facilities matching those in [ECMA-402](#).



- Tom returned to the subject of requirements and asked if there are cases where multiple locales need to be consulted for the same message.
- Markus responded that mixtures of locales appear in cases where a placeholder refers to, for example, a name.
- Steve stated that such scenarios are common with currency; it is common, for example, to use USD outside of US locales.
- Mihai added that this also happens with dates. Dates may be presented in multiple formats.
- Steve added that use of a locale independent date format might be consistently used regardless of locale.
- Mihai stated that these scenarios should be possible to address, but not optimized for.
- Tom asked if the notion of locale independent message formatting is relevant to the MFWG.
- Markus replied that there are good use cases for locale independent formatting; logging for example.
- Steve added that JSON output should not be localized.

## March 25th, 2020

### Draft agenda:

- Follow up on our meeting with the Unicode Message Format Working Group
  - Draft volunteers to function as liaisons between our groups.
- Plans to acquire implementation experience for:
  - [P1949R1: C++ Identifier Syntax using Unicode Standard Annex 31](#)
  - [P2071R0: Named universal character escapes](#)
- [D2124R0: std::regex should be deprecated starting in C++23](#)
  - Initial SG16 review.

### Attendees:

- Amanda Kornoushenko
- David Wendt
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann

### Meeting summary:

- Summer plans in lieu of the Varna meeting.
  - Tom introduced the topic.
    - With the cancellation of the Varna meeting, the library and language subgroups are going to start hosting telecons. This raises two concerns:
      - We'll face increased competition for our respective telecon time budgets.
      - We traditionally have not conducted polls during telecons, but we'll need to start doing so if we are to get papers forwarded to the subgroups for review during their telecons.
    - Tom also mentioned that mailings will now occur monthly.
  - Tom asked if anyone had concerns about continuing to attend SG16 telecons due to potentially having to choose between SG16 or other subgroup telecons. No concerns were reported.
  - PBrett asked how polling will be conducted.
  - Tom explained his current thoughts; we can adopt a tentatively ready approach and affirm decisions at the next face-to-face meeting.
  - PBrett suggested providing some method of async polling for those that are unable to attend telecons due to time zone or other challenges.
  - Tom replied that it is important that those voting are present for discussion leading up to a poll.
  - Mark stated that the current schedule is ok, but that we could consider setting aside two time slots favorable to different time zones and swap between them.
  - PBrett suggested that having a tentatively ready queue provides an opportunity for a two tier approach to decision making.
  - JeanHeyd suggested adopting the LWG approach of monday emails preceding a meeting; that would invite more participation from those that cannot attend a telecon.
  - Tom expressed support for that suggestion.
  - Mark opined that polling should not occur without an author present as we'd be unlikely to converge on consensus.
  - Mark suggested that, perhaps, we could record discussion.
  - Tom replied that he could follow up with Herb regarding that possibility.
  - PBrett suggested that we can publicize tentatively ready decisions, request feedback, and revisit based on new information.



- Hubert stated that meeting minutes are useful, but that he would object to sharing recordings since this group is open. Trust between committee members is built over time and we have to trust people to understand mis-statements.
- Tom asked Hubert that, if Herb were to approve some kind of recording, if this is something he could potentially be open to.
- Hubert responded with maybe, but only if a demonstrable need is recognized that can't be addressed in another way.
- Tom agreed with that criteria; recordings are sensitive, we can revisit this if need arises.
- Steve stated that more frequent mailings will help give us notice of poor responses to previous decisions. Writing a short paper with new information should not be a high bar.
- Hubert replied that mailings are a heavy weight way of providing feedback. During face to face meetings, we accept and address feedback in real-time; sometimes including in plenary.
- Tom agreed, but added that writing a paper is always an option.
- Steve expressed a preference that there be a relatively high bar for reopening discussions.
- Tom agreed and provided an example case where discussion was reopened. SG16 approved [P1885R0](#) in Belfast, discussion afterwards resulted in the paper being revisited in Prague where [P1885R2](#) was approved.
- Hubert stated that, since SG16 is a study group, objections can always be raised with EWG, LEWG, or in plenary.
- Tom returned to the mechanics of polling. Bluejeans doesn't have polling features built in. We could switch to Zoom so that we could make use of its "raise hand" feature. Otherwise, we'll have to figure out another way to conduct polls.
- Tom stated that he would document a process to use and we can evaluate consensus on it at an upcoming telecon.
- Follow up on the prior meeting with the Unicode Message Format Working Group
  - Tom asked if there were any volunteers to function as a liaison between our groups.
  - PBrett volunteered to do so.
- Plans to acquire implementation experience for:
  - [P1949: C++ Identifier Syntax using Unicode Standard Annex 31](#):
    - Tom asked Steve if he had any thoughts on implementation.
    - Steve responded that he had not strongly considered it, but that he didn't think it should be too difficult, at least for gcc where warnings are already emitted.
    - Steve added that the immediate priority is to free up time to answer new questions raised since the Prague meeting.
    - Tom suggested that it may suffice to audit the warnings that gcc provides and, if they closely match the proposal, to consider that sufficient implementation experience.
    - Steve stated that the complicated part is the filter for what characters are allowed.
    - Tom suggested another approach might be to analyze the gcc tests and add more as necessary to cover edge cases.
    - Steve responded that there is a table for allowed characters already and that he could probably evaluate that prior to the New York meeting.
  - [P2071: Named universal character escapes](#):
    - Tom stated that he has not started on an implementation yet; he is currently focused on getting implementations of `mbrtoc8` and `c8rtomb` ready to submit to glibc.
    - Tom added that he might start an implementation after that, but if anyone else wants to work on it, that would be great.
    - JeanHeyd asked if an implementation was needed given the prior work that Corentin did.
    - Tom responded that, technically no, but having an implementation available helps with consensus; it means we're standardizing an existing practice.
    - PBrett agreed; an implementation increases confidence in a design.
    - Hubert stated that the implementation impact for C vs C++ may be much higher; WG14 may object.
    - Jens noted that WG14 will want two implementations, but that acceptance in C++ counts as one.
    - JeanHeyd asked if anyone was aware of a simple C compiler that would be good for experimentation.
    - PBrett replied that [TCC](#) might be a good choice, but that it appears to be unmaintained.
    - Jens noted that the implementation doesn't require a modern C compiler.
    - JeanHeyd suggested that [SDCC](#) may be another option.
    - Steve stated that he would be surprised if there was significant code size impact as opposed to data size.
    - Jens responded that it doesn't much matter; growth is growth and potentially impactful for constrained use cases. Even 300K could be significant.
    - Steve provided an example impact; the compiler may no longer fit on a floppy disk.
    - Jens suggested it may make sense for this feature to be optional for C.
    - Jens noted that Richard Smith had asked about allowing `\N` in identifiers and suggested they should be allowed there.
    - Tom responded that it is on his todo list to address that in a revision. We'll then want to discuss that option at a future telecon.
    - Jens added that, from a core wording perspective, it would be simpler to specify these as *universal-character-name*; allowed uses could still be differentiated.
- [D2124R0: std::regex should be deprecated starting in C++23](#):
  - PBindels introduced the draft:
    - `std::regex` doesn't work with variable length encodings.
    - Performance is poor to the point it is faster to fork and exec a process to do the regex in another language.
    - We can't fix the problems due to ABI concerns.
    - There is no plan to remove; just to deprecate until a suitable replacement is provided, as was done for `std::auto_ptr`.
    - We don't want to spend more committee time on `std::regex`.
  - PBrett added that he has drafted additional changes that discuss what a new regex implementation could provide and had planned to target those changes to the Varna pre-meeting mailing.
  - Hubert noted that we now have monthly meetings.

- Tom noted that the next mailing deadline is now April 15th.
- Hubert stated that the relatively large set of supported RE grammars is a problem; we should avoid POSIX ones that require implementation-defined behavior.
- PBrett noted that this concern is mentioned in the paper, but needs more detail.
- PBrett also noted that the paper would benefit from a reference to [UTS #18](#).
- Tom suggested that, perhaps, a separate paper listing requirements for a `std::regex` replacement would be useful.
- JeanHeyd responded that is useful to have that information in this paper to guide authors who might like to propose a replacement.
- PBrett suggested that a requirements paper could be provided later.
- PBindels opined that direction for a replacement should be documented in a new paper.
- Steve suggested that inability to handle Unicode is reason enough to deprecate `std::regex`.
- PBrett noted that `std::regex` can be used for UTF-8 if the regular expression author is very careful and text is normalized.
- PBrett also noted that `std::regex` can be used on strings that are not text.
- PBindels added a comparison; just like `std::string` can be used for non-text.
- Mark gently steered the group back to the topic of deprecation.
- PBrett asked if anyone that regularly attends LEWG or LWG has additional guidance.
- Hubert responded that including relevant LWG issues would be helpful.
- Mark responded that it would be useful to include links to relevant papers that discussed issues with `std::regex` in the past and provided a list of such papers going back to 2005:
  - [P0169R0: regex and Unicode character types](#)
  - [P0014R1: Proposal to add the multiline option to std::regex for its ECMAScript engine](#)
  - [P0757R0: regex\\_iterator should be iterable](#)
  - [P1149R0: Constexpr regex](#)
  - [P1844R1: Enhancement of regex](#)
- Mark added that references to closed LWG issues would also be useful.
- Hubert agreed; the existence of significant issues may be sufficient motivation for removal.
- PBrett noted that relevant LWG issues can be found by searching for "regex" on the [LWG active issues list](#).
- Hubert provided a link to such an issue: <https://cplusplus.github.io/LWG/issue2546>.
- Jens noted that `export` was removed, so there is precedent for removal without replacement.
- Jens suggested that someone follow up with Peter Dimov since he was involved with `boost::regex` and the introduction of `std::regex` in C++11.
- PBrett suggested that, perhaps, the paper should include an option for removal.
- Jens cautioned against doing so.
- PBindels opined that there is plenty to gain by deprecation, but that noting the possibility of removal is an option.
- Jens added another prior deprecation example; `<sstream>` deprecation was not particularly difficult despite there still not being a full replacement for all use cases.
- JeanHeyd commented that [P0448](#) provides a span that is a full replacement for `<sstream>`, so removal may be possible relatively soon.
- Jens expressed a desire for an analysis of how a replacement could be evolved without running into the same ABI issues.
- PBindels agreed with that desire, but opined that should be done in a different paper.
- Tom provided a list of suggestions:
  - Drop all uses of "very", "much", "bad", "many", "entirely", "large", etc...
  - In 3.1.1, the example is good, but confusing. It may be worth emphasizing that use of braces denotes a set of individual code units (not characters!) to be matched. It may be helpful to explain the intent and the observed behavior separately.
  - In 3.1.2, it may be worth mentioning explicitly that Unicode has multiple ways to represent some characters.
  - In 3.1.3, it isn't stated what character is used as a space character in the example string.
  - Explicitly list the signatures of interfaces in `std::regex_traits` that are problematic. Having the signatures available makes some of the issues very clear. For example, what does it mean to translate a standalone code unit?
    - `CharT std::regex_traits<CharT>::translate(CharT c) const`
  - Explain why a replacement for `std::regex_traits` wouldn't suffice to address significant problems; the interfaces provided by `std::regex` are not problematic by themselves.
  - Reference failed papers.
- PBrett stated that people reading the paper should be aware of normalization.
- Jens responded that some won't and that they will form an opinion and vote on the paper regardless.
- Hubert asserted that the paper does not need to focus on details of Unicode; an additional problem is that `std::regex` doesn't differentiate between the encoding of the pattern and the subject.
- Hubert provided a link to a [gcc bug report](#) demonstrating that WG21 didn't understand the `std::regex` functionality when they standardized it.
- PBrett asked if anyone has major concerns about deprecating `std::regex`.
- Mark responded that, when his organization moved to C++11, `std::regex` was the only new feature programmers were told not to use.
- PBindels responded that he once tried to use `std::regex` to do an alternation on a few patterns and performance made it clear this wasn't going to work out.
- PBrett responded that his organization uses `std::regex` in production, in limited cases, with known Latin1 text.
- Hubert responded that he has come across cases where `std::regex` matching behavior is sensitive to compiler language dialect options.

- Steve responded that deprecating it would make code reviews easier since all uses of it could be rejected on the basis that deprecated features shouldn't be used.
- PBrett noted that `-wno-deprecated` is ubiquitous in their code base.
- Tom confirmed that the next meeting will be Wednesday, April 8th.

## April 8th, 2020

### Draft agenda:

- Discuss whether to hold telecons at the current UTC time year round and discontinue observing day light savings time.
- Discuss and poll to adopt the proposed SG16 operational plan:
  - <https://github.com/sg16-unicode/sg16/blob/78d6b4052ed561a6f5d384d6b5a4c7f30ac523c6/OperatingProcedures.md>
- Discuss whether to switch from Bluejeans to Zoom for future meetings.
  - WG21 is encouraging (but not requiring) all SGs to use Zoom for consistency and for polling features.
- Unicode Message Format Working Group liaison report.
- [D1949R3: C++ Identifier Syntax using Unicode Standard Annex 31](#):
  - New draft revision review.

### Attendees:

- David Wendt
- Hubert Tong
- Jens Maurer
- Mark Zeren
- Peter Brett
- Steve Downey
- Tom Honermann
- Zach Laine

### Meeting summary:

- Discussion of telecons and daylight savings time:
  - Tom introduced the topic.
    - We've historically observed daylight savings time as observed in the EST5EDT4 timezone.
    - Adjustments to meeting times due to timezone changes are a cause for confusion.
    - Proposing to keep telecon times at the same UTC time year round.
    - Any concerns?
  - Everyone tried to work out what local telecon times would be when timezone adjustments next occur. As might be expected, there was some initial confusion.
  - Jens cleared up the confusion; when timezone changes are made in the fall, telecons will start one hour earlier for locales that observe daylight savings time.
  - Mark indicated that the earlier time might make for some tight scheduled for him, but probably ok.
  - Tom stated that we'll try this and can always adjust if necessary.
- Discussion and polls to adopt the proposed SG16 operational plan:
  - Tom introduced the topic.
    - As discussed during our last telecon, changes happening in WG21 in response to the COVID-19 crisis will require that we begin polling papers during telecons.
    - Tom circulated a draft document with proposed procedures for SG16 telecons and meetings:
      - <https://github.com/sg16-unicode/sg16/blob/78d6b4052ed561a6f5d384d6b5a4c7f30ac523c6/OperatingProcedures.md>
  - PBrett asked for clarification regarding forwarding of papers and whether a requested change means that we'll need to wait for an updated revision and re-poll.
  - Tom responded, no; we can approve with feedback and audit the next revision to ensure that our feedback was addressed. Such auditing may require naming a delegate to follow up in some cases.
  - Steve noted that the isocpp.org paper management system allows for P-numbered papers that have not yet appeared in a mailing to be updated. Such updates should be avoided for P-numbered papers that have been shared prior to appearance in a mailing.
  - Jens stated that we should strive to avoid revision inflation but that it is probably beneficial to have a P-numbered revision when polling to forwarding a paper to an upstream subgroup.
  - **Poll: Adopt the proposed document as SG16's operating procedures**
    - **Attendees: 8**
    - **No objection to unanimous consent.**
- Discussion of whether to switch from Bluejeans to Zoom for future meetings.
  - Tom introduced the topic:

- WG21 leadership has suggested that all subgroups and study groups adopt Zoom for their telecons for consistency, familiarity, and polling features.
- SG16 has been using Bluejeans successfully for the last two years.
- We haven't encountered many technical issues with Bluejeans.
- Bluejeans doesn't provide polling features. Now that we'll need to be polling in telecons, such features could prove helpful.
- Should we switch to Zoom?
- PBrett noted that there have been reports of serious privacy and security issues with Zoom.
- Tom responded that Herb has provided guidance on how to configure Zoom to address some of these issues.
- PBrett added that there are also concerns regarding how Zoom makes money.
- Tom responded that the ISO funds accounts for chairs that need them. In my case, my ISO registration is via my work email address and I have a Zoom account funded by my employer.
- PBrett noted that WebEx doesn't monetize usage monitoring; Zoom seems to have a we-can-do-whatever-we-want approach to usage monitoring.
- Jens responded that eavesdropping concerns are probably not a strong concern for the ISO. And SG16 is a public group.
- Tom agreed.
- Jens stated that he has been having technical issues with Bluejeans; it fails to populate the participant list and chat. No such issues have been experienced with the Zoom client.
- Tom asked if there were any strong objections to switching to Zoom.
- PBrett responded that, if the question was phrased as, would I stop attending SG16 telecons if we switched to Zoom, then the answer is no.
- Tom restated the question to just ask for preferences.
- Zach responded that he has no preference; both Bluejeans and Zoom work for him.
- Mark responded likewise.
- Steve responded that he has both installed and noted that the moderation features are better in Zoom. If SG16 were a larger group, we would need such features, but Bluejeans works fine for the size of our group.
- Zach stated that the LEWG telecon hosted with Zoom this week worked pretty well. There were some challenges getting everything on the screen; chat was pretty active, the raise hand feature was being used and that required the participant list. He had to tile windows to make everything fit. The hand raise feature was nice. The LEWG chair struggled a little bit keeping tabs on chat, hands, screen.
- Zach added that he wasn't sure the raise hands feature is needed for groups of our size.
- PBrett reported having had a similar experience; it can be challenging to use Zoom without using multiple monitors.
- Mark noted additional issues with the Zoom UI layout; widgets may obscure the main window.
- PBrett stated that we should have more motivation for switching.
- Jens reminded the group that Bluejeans is not working correctly for him.
- Tom suggested that we wait a month or so to evaluate how things go with other groups and then revisit.
- Unicode Message Format Working Group liaison report.
  - PBrett reported that, like everyone, they have been impacted by the COVID-19 pandemic, but they have worked out a system for rotating chairs for meetings.
  - PBrett added that, if anything interesting happens, that he will notify Tom to put an item on the agenda; there is no reason to have a liaison report at each of our meetings at this time.
- [D1949R3: C++ Identifier Syntax using Unicode Standard Annex 31](#)
  - Steve introduced changes since [P1949R2](#):
    - The most significant changes relate to when normalization occurs.
    - There is implementation divergence with regard to preprocessor identifiers.
    - gcc requires that preprocessor identifiers meet the general identifier requirements, but Clang and Visual C++ do not. gcc rejects the following code because `\u0300` is not a valid initial character for an identifier.

```
#define accent(x) x##\u0300
constexpr int accent(A) = 2;
constexpr int gv2 = A\u0300;
static_assert(gv2 == 2, "whatever");
```

- PBrett expressed a preference for gcc's behavior.
- Tom stated that the operands to the `##` operator are not identifiers.
- Jens agreed; they are preprocessing tokens.
- Steve responded that the grammar for preprocessing tokens includes identifier.
- Hubert agreed; the grammar for preprocessing tokens is stated in terms of identifiers.
- PBrett: provided a link to the grammar for *preprocessing-token* showing identifier.
  - <http://eel.is/c++draft/lex.ptoken#nt:preprocessing-token>
- PBrett asked if it would be reasonable to modify the preprocessing grammar to require adherence to [UAX #31](#).
- Hubert responded that the question previously raised on the mailing list was, after token pasting, is it required to check that a resulting preprocessor token that is ostensibly an identifier is in NFC and, if not, is the result undefined-behavior (UB); in other words, is a non-NFC identifier a valid preprocessing token?
- [ Editor's note: that question was raised in the email thread that started at <https://lists.isocpp.org/sg16/2020/02/1122.php>. ]
- Zach stated that, in general, yes, checking is required because combining two NFC sequences doesn't necessarily produce an NFC sequence. If we're going to diagnose ill-formedness, we need to check for that. Otherwise we end up with ill-formed-no-diagnostic-required (IFNDR) and that should be avoided.

- PBrett mentioned that the draft paper doesn't have an example of pasting two NFC tokens that produce a non-NFC result.
- Steve responded that the example in the paper is such an example.
- [ *Editor's note: The example in the paper is such an example, though not necessarily the kind of example that Zach and Peter had in mind. Including an example from the [UAX #15 section on concatenation of normalized strings](#) might be helpful.* ]
- Jens noted that the second operand is not a valid UAX #31 identifier.
- Steve acknowledged that, but noted that a digit is not an identifier, but can be concatenated to a valid identifier to produce a different one.
- PBrett asked if a bare combining character can be a valid preprocessing token; whether, in C++20, `\u0300` is a valid preprocessor token.
- Jens responded, no; [\[lex.name\]p1 table 3](#) lists U+0300 in the list of combining characters that are not permitted to start an identifier.
- PBrett summarized; Clang and Visual C++ are non-conformant because they allow `\u0300` as a preprocessing token, but gcc handles this correctly. This is the status quo.
- Jens stated that the lexer permits much undefined behavior, so we need to be careful.
- PBrett agreed and added that, for cases of existing UB, we can change behavior.
- Jens opined that the example ought to be ill-formed, but that there are oddities in the lexer and preprocessor specifications. For example there is wording that if the result of a concatenated token is not a valid preprocessor token, then the result is UB; would like this example to be ill-formed.
- PBrett asked if this issue should be reviewed by SG12.
- Jens responded, no; so long as we're not removing allowances for UB or adding new UB, SG12 doesn't need to be involved. The example appears to be ill-formed.
- Tom asked for clarification; Clang and Visual C++ are missing a diagnostic?
- Jens responded, yes.
- Zach stated that this suggests that the NFC check should be performed later in translation as opposed to checking that each operand of the `##` operator is in NFC.
- Jens disagreed with that approach.
- Steve observed that there are interesting interactions with header units since they externalize preprocessor macros and require comparisons of them.
- Tom opined that the issue applies equally for headers since different header files can have different source encodings.
- Jens suggested that implementations could convert to NFC as part of translation phase 1.
- Hubert reported having investigated implementation divergence and found that there seems to be confusion regarding how max munch works for identifiers. [preprocessing-token](#) has a rule to match non-white-space characters that don't otherwise fit in the grammar. Max munch fails to consume a `\u0300`, so `\u0300` is a valid preprocessing token.
- Jens noted the change in perspective; so the example is well-formed and gcc is wrong to reject it.
- PBrett concluded that, if `\u0300` is a non-white-space preprocessing token and is therefore a valid operand for the `##` operator, then that means that the earliest we can diagnose non-NFC identifiers is after token pasting.
- Steve summarized that the diagnostic options are translation phases 4 and 7.
- Hubert stated that there is a difference between writing `\u0300` and the actual U+0300 character in the physical file.
- [ *Editor's note: Hubert clarified after the call that he was speaking to a need to consider the user expectations in both cases. The conversation got away before this point could be clarified.* ]
- Jens responded that, in translation phase 1, all extended characters are converted to *universal-character-name*; bare character don't exist afterwards.
- Tom noted that differences are observable in raw literals.
- Steve acknowledged, but noted that isn't relevant for identifiers.
- Zach stated that Hubert's point is important; macros need to be NFC checked. But we want to make things easy for implementors. In the example, if we wanted to check the result of the concatenation in translation phase 7, is there a simple rule to just check for macro names? Or a simple rule to check all identifiers in translation phases 4 and 7? It may be worth asking implementors for opinions.
- PBrett stated that the check should be performed at the point that something becomes an identifier. That solves the problem for `#define` since it requires an identifier.
- Hubert responded that he is ok with the lexing portion of that. The question is, when you try forming a preprocessing token and it is supposed to be an identifier, do you need to check again? If not, then further checking is needed when converting preprocessing tokens to tokens. Deferral would be preferred in order to avoid interaction with the UB that Jens pointed out regarding token pasting producing invalid preprocessing tokens.
- Zach summarized; we do need to check that preprocessing tokens used as identifiers are NFC. And we need to check that the result of token pasting is a NFC preprocessing token if used as an identifier. And we need to check identifiers at translation phase 7.
- Jens replied that the wording we have is already what we want; it doesn't differentiate between preprocessor or translation phase 7 identifiers. The result of token pasting is just a preprocessing token, so there is no need to perform an NFC check since it isn't an identifier yet.
- Hubert clarified his understanding of Zach's description; Jens stated that the result of token concatenation is just a preprocessing token, but it can be any of the preprocessing token possibilities. If it matches an identifier, then we can check if it is NFC.
- PBrett stated that, in [\[cpp.concat\]p3](#), if the result is not a preprocessing token, then the behavior is UB. So we don't have to check for NFC because it is already UB.
- Hubert responded that the question is then whether we care for this use case.
- Jens noted that caring for this use case is more expensive.
- PBrett stated that the case he is concerned about is where token pasting produces a name that is then used to define a macro; a diagnostic should be issued in such cases.
- Jens responded that UB has already occurred at that point, so a diagnostic doesn't apply.

- Jens noted that the only use cases we have for producing non-NFC preprocessing tokens are for use with the stringize operator
- Hubert added, or for discarding tokens. The stringize case isn't too compelling because regular string concatenation suffices.
- Hubert brought up another concern that had been previously raised; *preprocessing-token* includes *pp-number*. For user defined literals (UDLs), if we only check the identifier grammar, then we only check at phase 7 when these tokens become UDL suffixes.
- Steve observed that this is where the discussion regarding use of currency characters in UDLs comes into play.
- Jens stated that the current rules for [\[lex.name\]](#) are reasonable and good enough since they restrict what identifiers can be.
- Zach asked if the implication is that we need to check for NFC at translation phases 4 and 7.
- Jens responded, yes.
- PBrett added that the example is then ill-formed because substituting the accent in the first `constexpr` line produces UB, and the second `constexpr` is ill-formed because `A\u0300` is not in NFC.
- Steve stated that he will try to update the paper to clarify this.
- *[ Editor's note: Several minutes of discussion were not recorded because the editor was having a hard time following it. ]*
- Jens stated that we may need to introduce a *pp-identifier* preprocessing token kind to replace use of *identifier*; the NFC check would then happen when converting a *pp-identifier* to an *identifier*.
- PBrett expressed support for this direction as it avoids the UB and difficulties with lexing the source code.
- Jens asked Hubert if he is content with the introduction of a new *pp-identifier* term.
- Hubert responded that he is.
- Steve summarized the direction. A new *pp-identifier* will be introduced that matches the preprocessor notion of an identifier and for which an NFC check will be performed at the point that it is converted to an identifier. This avoids potentially needing to perform incremental NFC analysis during lexing.
- PBrett asked if the new *pp-identifier* could require conformance with UAC#31, but just not be in NFC.
- Zach replied, yes.
- PBrett asked for confirmation that a *pp-identifier* will never need to be compared.
- Jens replied, correct.
- Hubert added that this retains the case where a lone `\u0300` is neither a *pp-identifier* nor an *identifier*; it is one of those lone non-white-space preprocessing tokens.
- Jens confirmed; right, because a *pp-identifier* must start with an `XID_Start` character.
- Steve asked for confirmation that *pp-identifier* will appear in the grammar roughly like *pp-number* does.
- Jens confirmed, yes.
- Tom asked Steve if additional feedback is needed.
- Hubert asked if there is any further question about how to apply the UAX #31 conformance statements.
- Steve replied that he would welcome recommendations on how to present that. At present, the paper documents a profile for conformance with R1, and an NFC requirement for conform to R4. The remaining requirements are intentionally unmet or not applicable.
- Jens replied that the phrasing doesn't seem right. The intent of UAX #31 is to require documentation stating what requirements apply and why or why not; the paper is lacking some introductory text.
- Steve acknowledged, something along the lines "profile is, ..., and the others are not applicable".
- Jens requested that the paper be updated to use complete sentences.
- Steve agreed to do so.
- Tom noted that, in section 9.3.1, there appears to be a formatting issue.
- Steve acknowledged and stated he would correct it.
- Jens volunteered to follow up with suggested wording on the mailing list.
- *[ Editor's note: Jens did so; the email thread is available at <https://lists.isocpp.org/sg16/2020/04/1235.php>. ]*
- *[ Editor's note: After the meeting, Hubert sent a message to the mailing list arguing that attempted concatenation of `\u0300` concatenates only the `\` character because the lexer observes `\`, `u`, `0`, `3`, `0`, `0`, not the single universal-character-name. The result is UB because the concatenation doesn't produce a valid preprocessing token. The email thread is available at <https://lists.isocpp.org/sg16/2020/04/1229.php>. ]*
- Tom reminded the group that WG21 is moving to monthly mailings and that the next mailing deadline is April 15th.
- Tom confirmed that the next meeting will be on April 22nd.

## April 22nd, 2020

### Draft agenda:

- [Core issue 1871: Non-identifier characters in ud-suffix:](#)
  - [SG16 github issue #61](#)
  - Core decreed that this issue is evolutionary. JF requested that SG16 review and provide a recommendation.
- [P1949R3: C++ Identifier Syntax using Unicode Standard Annex 31](#)
  - Review updates since the April 8th review.

### Attendees:

- Hubert Tong



- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann

## Meeting summary:

- [Core issue 1871: Non-identifier characters in ud-suffix](#)
  - Tom introduced the topic.
    - This is a Core issue that has been deemed evolutionary and sent to EWG to handle.
    - JF requested that SG16 provide a recommendation.
  - PBrett expressed distaste for the idea and that it should be rejected. ISO 4217 provides a standardized set of currency identifiers that avoid challenges imposed by symbols. We could instead define a currency language or library facility.
  - PBindels agreed.
  - Jens agreed as well and noted that the ISO 4217 specification is what the finance community depends on.
  - PBrett stated that we don't want to allow use of symbols that we might want to use for non-identifier things like operators in the future.
  - PBindels noted that some currency symbols already serve other purposes. For example, some compilers support an extension allowing use of \$ in identifiers.
  - Tom added that some currency symbols are overloaded; \$ doesn't mean USD.
  - Steve added that there may be aliasing issues with legacy character sets.
  - Mark suggested it might be worth getting input from Mateusz Pusz given his work on [P1935](#) and strong types and UDLs for physical units.
  - Jens responded that we need not spend further time soliciting input from non-SG16 attendees. EWG will handle this; our responsibility is to provide the SG16 consensus.
  - **Poll: Is there any objection to unanimous consent for recommending rejection of this proposal?**
    - **Attendees: 7**
    - **No objection to unanimous consent.**
- [P1949R3: C++ Identifier Syntax using Unicode Standard Annex 31:](#)
  - Tom introduced the topic:
    - Steve submitted an updated revision for the April mailing that addresses the feedback provided in our [April 8th review](#).
  - Steve summarized the changes:
    - *pp-identifier* was introduced to allow preprocessing tokens that are ostensibly identifiers to be non-NFC until converted to an identifier. At that time, a well defined event during translation, NFC would be checked.
    - This approach allows non-NFC identifier-like tokens to be used in locations that aren't used as identifiers; for example, when stringizing tokens.
    - This approach does permit the possibility of aliasing in some scenarios, but those cases should be handled as don't-do-that.
  - Hubert objected that, with respect to macro invocations, aliasing can lead to surprising behavior.
  - PBrett asked if the paper didn't already address that.
  - Jens clarified Hubert's objection; it is undisputed that a macro definition requires an NFC identifier, but when scanning macro text, a non-NFC *pp-identifier* intended to name a macro can't be diagnosed.
  - Steve stated that Hubert provided a good example on the SG16 mailing list.
  - [ Editor's note: That example is in <https://lists.isocpp.org/sg16/2020/04/1259.php>. ]
  - Jens summarized the behavior of that code. Depending on normalization, the result is either the stringized form of the macro replacement text or something else.
  - PBrett noted that, in this case, the check for NFC was evaded by use of the stringize operator.
  - Steve stated that he won't claim that there aren't other ways that this can happen, but in most cases, this will eventually result in a syntax error.
  - Jens noted that lone combining marks might cause odd editor behavior depending on what precedes them.
  - Tom stated that there are two ways to attack this problem. Either we require *pp-identifier* tokens to be NFC, or we delay the check until they are used as identifiers.
  - PBrett responded that the second approach leads to the problems we discussed at the last telecon.
  - Jens noted that, for `\u0300`, since it doesn't constitute a valid identifier due to U+0300 not being in the set of characters allowed initially according to [\[lex.name\]p1](#), `\` is one token and `u300` is another. This is the status quo and can lead to tearing of combining characters.
  - PBrett asked if this issue can be dodged by changing stringization such that *pp-identifier* cannot be stringized unless it is also an identifier.
  - Hubert responded that a narrow approach probably isn't desired. The rationale is that, if you take *pp-identifier* and don't enforce NFC, `XID_Start`, or `XID_Continue` on it, if lexing happens a certain way, then we won't be able to adopt any Unicode characters as new operators without impacting backward compatibility.
  - Jens added, for example, the sigma character.
  - Hubert opined that infix operators are more compelling. For example instead of the `|>` operator currently being discussed.
  - Jens summarized the problem Hubert indicated; in the paper as worded, symbols can be lexed into single tokens that can then be concatenated. That's bad.



- Jens added that this suggests the proposed *pp-identifier* approach fails in the long term and that a more narrow fix is needed; `XID_Start` and `XID_Continue` should be enforced for *pp-identifier*.
- Steve noted that there are combining marks in `XID_Continue`, even in NFC.
- Jens stated that we don't want to enforce NFC during character-by-character lexing.
- Jens suggested another approach. The idea is to require `XID_Start` and `XID_Continue` for lexing of *pp-identifier* and then to do the NFC check on the resulting token. That addresses the case where a *pp-identifier* in macro replacement text might name a macro or macro parameter.
- Jens continued; the case that must be avoided is the lone combining mark. That can be addressed by adding a *pp-lone-ucn* to *preprocessing-token* and that can be used with the concat operator with the result that doing so will lead to a diagnosable error later if used as an identifier. The max munch rule would apply first so that *universal-character-names* that specify a combining character in `XID_Continue` and are preceded by a valid identifier are incorporated into the identifier.
- Hubert asked if production of such a token would be diagnosed immediately.
- Mark asked for clarification; whether *pp-lone-ucn* could begin with a character not in `XID_Start`.
- Hubert followed up with an additional question; in the case of two consecutive not visibly separated UAX #31 identifiers; is that two preprocessing-tokens?
- Jens responded that a sequence of characters that are only in `XID_Continue` would each be individual tokens.
- Jens directed the group to wording provided on the wiki; there is a preexisting defect that we should not try to fix in this effort, but which I think we can.
- [ Editor's note: For those with access to the WG21 wiki site, that wording is [an attached file on the summer 2020 SG16 project page.](#) ]
- Steve asked if this design would prohibit stringizing a lone combining character.
- Jens responded that it would and that lone combining characters in source code are bad.
- Steve agreed; if you want to combine a combining character, use a string literal.
- PBrett agreed as well and stated he would have serious questions about such a use case.
- Tom asked for confirmation that this approach applies equally regardless of whether the combining character appears in the source file as an extended character or as a *universal-character-name* escape.
- Steve confirmed.
- Jens stated that there is a remaining issue that his proposed wording removed *identifier-non-digit* but that there are now dangling references to it in the definition of *pp-number*.
- Hubert stated that the usual characteristic of *pp-number* that is of interest here is for UDLs; *pp-number* describes how you get UDL suffixes as identifiers.
- Steve stated that *identifier-non-digit* is *non-digit* or *universal-character-name*, but *universal-character-names* in identifiers can not specify a character from the basic source character set.
- PBrett asked if Hubert is more comfortable with Jens proposed changes and whether we'll need to discuss this again.
- Hubert responded that he is happy with this and confirmed the presence of wording that makes lone UCNs ill-formed, the wording that enforces `XID_Start` and `XID_Continue` for *pp-identifier*, that the basic source character details are handled by the existing non-terminals, and that tokens look perfectly safe.
- Jens indicated that the wording edits to *pp-number* are now present and for viewers to reload the wiki page.
- Hubert stated that, as wording goes, this is pretty natural. It covers corner cases, but doesn't require that the wording call them out.
- Jens suggested that it may be worth noting in the annex how our grammar relates to the UAX #31 grammar; that our non-digit terminals are all in `XID_Start`.
- Hubert suggested adding wording to explicitly avoid UB on token concatenation by ensuring that the program is ill-formed if token concatenation produces a non-NFC token.
- Mark asked about an update to 5.4p2 and whether "single" was stripped from "non-whitespace" character.
- Jens indicated it should apply to both.
- Mark acknowledged.
- Hubert stated that "the last category" wording in 5.4 should be updated since it doesn't apply to just the last category any longer.
- Jens updated the wording and indicated to reload the page.
- PBrett asked whether, in Jen's new wording, if the reference to UAX #44 for `XID_Start` and `XID_Continue` should be to UAX #31?
- Steve responded, no.
- Jens explained that the `XID_Start` and `XID_Continue` properties are defined in UAX #44; UAX #31 specifies the requirements for how to use them.
- Hubert expressed sympathy for trying to name these categories; e.g., *stray-universal-character-name*.
- Jens indicated he presumes the wording is generally ok with folks now.
- Jens stated that he sent around an email with other comments.
- [ Editor's note: That email can be found at <https://lists.isocpp.org/sg16/2020/04/1256.php>. ]
- Steve acknowledged and stated that he would respond and that he agreed with the suggestions.
- Jens noted that some of those concerns have been addressed by discussion today.
- Jens stated that there should be a statement about the difficulty of checking for NFC and asked if we can qualify the difficulty for matching `XID_Start` and `XID_Continue`; "gcc already does this" doesn't provide much assurances.
- Hubert suggested that Steve may want to ask Hal about fixing the paper submitted for the mailing to remove the draft indicator in the paper's header.
- PBrett asked if we will poll the paper today.
- Tom responded, no; given discussion and new wording, that he would like discussion from today to sit in our minds for a few weeks and be incorporated in a new revision before we poll.

- Jens noted that his wording avoids adding a normative reference to UAX #31; only a normative reference to UAX #44 is actually needed.
- Steve acknowledged; the non-normative reference and annex just provides rationale for the design intent.
- Tom presented some additional suggestions for the paper:
  - Additional rationale:
    - Motivation: Note potential security concerns; that misbehaving code may pass code review.
    - Legacy encodings convert naturally to NFC.
    - Programmer and text editors tend to produce NFC; can that claim be backed up with some data?
    - Discuss translation phase 1; since this is implementation-defined, implementations can convert to NFC. However, doing so may interfere with author intent since non-NFC is permitted in string literals.
  - Wording:
    - Include removed wording so that readers can see what is removed without having to go look it up.
    - Typo in X.3 R2, Immutable Identifiers; "idenfiers".
    - Typo in X.4 R3, Pattern\_White\_Space and Pattern\_Syntax Characters; "properites"
- Hubert asked if we want to discourage implementors from doing NFC conversion in translation phase 1.
- Jens responded that doing anything to phase 1 will impact code compatibility. It would be ok to discuss this in front matter; that if you do NFC conversion in translation phase 1, that you may want to consider string literals.
- Mark observed that if one compiler does NFC conversion in translation phase 1 and another one doesn't, then the source file is non-portable.
- Steve responded that source encoding is always non-portable.
- Jens stated that he just doesn't want any normative wording changes for translation phase 1.
- Steve asked if we'll plan to discuss this again.
- Tom responded, yes.
- Mark suggested the next discussion of it should be relatively short; unless Hubert finds more interesting examples!
- Steve stated that he will proceed with making changes and asked Jens if he is satisfied with the wording currently in the wiki.
- Jens replied that he is.
- Tom indicated that the next telecon will be by May 13th; three weeks from now.
- Steve noted that will be just in time for the next mailing.

## May 13th, 2020

### Draft agenda:

- Review the queue for C++23:
  - How are we doing relative to the directives in [P1238R1](#)?
  - What is our vision for C++23? (What would be our elevator pitch?)
  - What features are we on track to deliver?
  - What features need additional prioritization?

### Attendees:

- Corentin Jabot
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Peter Bindels
- Peter Brett
- Tom Honermann
- Zach Laine

### Meeting summary:

- Review the queue for C++23:
  - Tom introduced the topic. Now that C++20 is complete, we have about two years until feature freeze for C++23. This is a good time to step back, review our projects in flight, determine which projects are on track for adoption in C++23, which ones we would like to get in C++23 but may be at risk of being ready in time, and which ones are not likely candidates for C++23.
  - PBrett asked where our queue of C++23 proposals can be found.
  - Tom replied that it is an ethereal container.
  - PBrett asked if it should be made more corporeal.
  - Tom replied that it should be.
  - Tom shared that he had reviewed [P1238R1](#), mapped active SG16 papers to each of its directives, and posted it to the SG16 Slack channel.
  - [ Editor's note: That map is below augmented with additional entries discussed during the telecon.

- 5.1: Standardize new encoding aware text container and view types
  - [P1629: Standard Text Encoding](#)
- 5.2: Standardize generic interfaces for Unicode algorithms
  - [P1628: Unicode character properties](#)
- 5.3: Standardize useful features from other languages
  - [P2071: Named universal character escapes](#)
- 5.4: Improve support for transcoding at program boundaries
  - [P1275: Desert Sessions: Improving hostile environment interactions](#)
  - [P1885: Naming Text Encodings to Demystify Them](#)
- 5.5: Propose resolutions for existing issues and wording improvements opportunistically
  - [P1949: C++ Identifier Syntax using Unicode Standard Annex 31](#)
  - [P1854: Conversion to execution encoding should not lead to loss of meaning](#)
  - [P1859: Standard terminology for execution character set encodings](#)
  - [P1880: uNstring Arguments Shall Be UTF-N Encoded](#)
  - [P2029: Proposed resolution for core issues 411, 1656, and 2333: numeric and universal character escapes in character and string literals](#)

]

- Tom continued; this provides some perspective regarding where we have been spending our time. Interestingly, the place we seem to be spending the most time, at least by paper count, is addressing existing core and wording issues. We have previously discussed those being lower priority objectives relative to adding new features.
- Tom started walking through the directives and associated papers.
- 5.1: Standardize new encoding aware text container and view types
  - [P1629: Standard Text Encoding](#):
    - Tom stated that he is feeling some concern about getting this through the committee process with just two years until feature freeze.
    - Corentin continued that thought; we don't have implementation experience yet. Getting this through the committee in two years seems possible. Getting it through in two years with everyone happy about it doesn't seem possible.
    - PBrett noted that JeanHeyd has been working on an implementation.
    - Tom added that JeanHeyd has been laying groundwork for the feature, particularly in WG14 for C.
    - Zach stated that his primary concern is collecting user experience feedback.
    - JeanHeyd described the current state of his work. He is working on new `mc` and `mwc` interfaces for WG14 for conversion between UTF-8, UTF-16, UTF-32, and the locale dependent narrow and wide character sets. These interfaces are designed similarly to `iconv` with hooks for accommodating private encodings and would be used to implement the fast path conversion implementations.
    - JeanHeyd stated that the implementation is working and the next step is wording for WG14.
    - PBrett expressed uncertainty regarding what the expectation is for C++23.
    - JeanHeyd responded that the bare minimum is support for encoding concepts and objects. [P1629](#) reflects this bare minimum; the encoding objects and associated free functions.
    - PBrett asked if there are dependencies between P1629 and the WG14 focused work.
    - JeanHeyd replied, no, they are distinct.
    - PBrett asked if it is practical to implement P1629 without the WG14 interfaces in place.
    - JeanHeyd replied yes, the implementor would just have to use non-standard encoding, decoding, and conversion routines.
    - Mark asked if implementors could provide an implementation based on `iconv`.
    - JeanHeyd replied, yes, but with the caveat that it would perform well for contiguous ranges, but not for highly segmented sequence containers like `std::list`.
    - Jens re-phrased Mark's question; the question was whether an `iconv` based implementation can handle something terrible like `std::list<char>`.
    - JeanHeyd responded, yes, but a temporary buffer would be needed. The buffer could be stack allocated.
    - Jens asked for verification that `iconv` can support code unit at a time conversions.
    - JeanHeyd replied that `iconv` takes pointers to in and out buffers and updates them to reflect the conversion state; unused code units can be cached.
    - Zach stated that `iconv` is only interesting as a proof of concept; users won't accept it due to poor performance. Bob Steagal showed that `iconv` can be badly beat by optimized conversion facilities.
    - Mark noted his intention in asking the question; whether implementors can reasonably provide a non-performant implementation to start with.
    - Tom observed that doing so could result in an implementor being stuck with a non-performant implementation due to ABI concerns.
    - JeanHeyd responded that, so long as state representation doesn't change, ABI shouldn't be an issue.
    - Tom noted that calls to `iconv` would appear in instantiated templates, so any changes to definitions of function templates or templated member function would raise ABI concerns; we don't want another `std::regex` debacle.
    - Zach re-iterated his desire to gather user experience.
    - JeanHeyd responded that getting an implementation in front of users is the goal of his current efforts, but that it will likely take until November to get the implementation fully in place.
    - Zach stated that implementation experience is great, but what he really wants is feedback from users since that is how we'll find the usability problems.

- Mark stated that it would be great to replicate the evolution that the `{fmt}` and `chrono` libraries followed, but acknowledged that this feature doesn't have quite the same kind of broad applicability.
- JeanHeyd responded that he has been contacted by programmers that are interested in using this.
- PBrett expressed skepticism that we'll be able to get this in for C++23 this way and that the best approach might be to get it accepted into Boost first.
- Mark suggested that Boost may not be the right vehicle for this.
- Zach opined that a standalone library with a couple of hundred stars would suffice; getting a library accepted in Boost takes time.
- Tom expressed a desire to get an implementation in front of users sooner than that.
- 5.2: Standardize generic interfaces for Unicode algorithms
  - [P1628: Unicode character properties](#):
    - Tom noted that we haven't talked about this proposal for a while.
  - Tom stated that he was hoping Zach, as one of few programmers that has actually implemented the Unicode algorithms, might be able to help make progress here.
  - Zach responded that he is hoping to get the ball moving on Boost.text again soon with a goal of getting more input and hopefully starting on papers by the end of the year.
  - Zach noted that `text` and `text_view` are somewhat novel, so that makes them risky for C++23.
  - Zach added that he did make some changes recently and that the algorithms are now faster than ICU except for tailored collation.
- 5.3: Standardize useful features from other languages
  - [P2071: Named universal character escapes](#):
    - Tom noted that this paper was received well by EWG in Prague and seems on track for C++23.
    - Tom stated that he has some minor updates to do to the paper before getting it back in front of EWG again.
  - Tom asked if there are other features we should be focusing on.
  - PBrett responded that he is aware of other features, but that they don't really fit into the C++ standard library.
  - Zach asked what features Peter had in mind.
  - PBrett responded that some languages have distinct string types for different kinds of strings. For example, Rust has OS strings.
  - Tom mentioned the encoding pragma paper that he has yet to write.
- 5.4: Improve support for transcoding at program boundaries
  - [P1885: Naming Text Encodings to Demystify Them](#):
    - Tom noted that this one has been making progress and that he would follow up with Corentin to inquire about next steps.
    - *[ Editor's note: Tom reached out and Corentin responded that he is concerned about the relatively weak support for the paper as is within SG16 and has some uncertainty regarding next steps. Tom encouraged updating the paper to document use cases and to compare IANA encoding representation with encodings supported by ICU, Microsoft, and the Encoding Standard to quantify representational deficiencies. ]*
  - [P1275: Desert Sessions: Improving hostile environment interactions](#):
    - Tom noted that this paper has languished and asked if anyone would like to champion moving something forward with respect to environment variables and command lines.
    - JeanHeyd stated that he will reach out to Isabella to find out what the status is.
    - PBrett noted that this is relevant for the [P1750](#) process invocation paper.
    - Tom agreed and remembered that Jeff mentioned in Prague that he and/or Elias were planning to split this functionality out to a new paper. Tom stated he would follow up with them.
    - *[ Editor's note: Tom reached out and Jeff confirmed intent to work on this, possibly within the next couple of months. ]*
- 5.5: Propose resolutions for existing issues and wording improvements opportunistically
  - [P1949: C++ Identifier Syntax using Unicode Standard Annex 31](#):
    - Tom noted that this paper is on track for C++23.
  - [P1854: Conversion to execution encoding should not lead to loss of meaning](#):
    - Tom stated that this paper was last [discussed in Belfast](#) and there are some concerns to be discussed and/or addressed.
    - Tom stated that he is not sure what Corentin's intentions are.
    - *[ Editor's note: Tom reached out and Corentin responded that, per the Belfast discussion, progress on this paper is blocked by dependence on P1885. Corentin is interested in revisiting the decision to take that dependency. ]*
  - [P1859: Standard terminology for execution character set encodings](#):
    - Tom noted that there hasn't been any movement on this paper since it was [discussed in Belfast](#).
    - PBrett expressed support for making this a top priority item to address.
  - [P1880: uNstring Arguments Shall Be UTF-N Encoded](#):
    - Zach stated that this paper is DOA; an audit of wording revealed 250 places where we would have to note an exception to front matter wording and having to do so makes this effort not worthwhile.
    - JeanHeyd asked if there are many interfaces that accept a `std::u8string`.
    - Zach responded that yes, there are due to many interfaces that accept a `std::basic_string`.
    - Jens elaborated; these are cases where the character type is determined by a distinct template parameter.
    - Jens stated that he would like to see an example where the encoding is relevant.
    - Zach replied that one of the places is the entire interface of `std::basic_string` itself; many such member functions accept a `std::basic_string`.
    - Jens noted that `char8_t` was invented to ensure a specific encoding.
    - Mark noted that many of these interfaces don't lend themselves to enforcing encoding invariants; `std::basic_string` wasn't designed to do so.

- Jens observed that the general provision that is desired is that `std::basic_string` objects used elsewhere meet such variants.
- Jens stated that hooks are needed in EWG and LEWG to get SG16 involved when certain topics come up.
- Tom replied that those hooks are in place, but with recent chair changes, should be re-iterated. Tom stated he would follow up.
- [ Editor's note: Tom did so and the EWG and LEWG chairs confirmed their intent to abide by [P1253](#). <https://lists.isocpp.org/sg16/2020/05/1292.php> ]
- [P2029: Proposed resolution for core issues 411, 1656, and 2333; numeric and universal character escapes in character and string literals](#):
  - Tom stated that this is on track and making its way through Core.
- Tom asked the group for opinions on what we should focus on for C++23.
  - PBrett opined that the papers under 5.5, and terminology updates specifically, should be top priority.
  - Zach agreed regarding terminology and added Unicode algorithms.
  - Mark opined that having text in Boost would be a big deal.
  - PBrett stated that SG16 needs to stay involved with [P1750](#), the process management paper. And we need to address how programs receive command line arguments.
  - Tom stated that, if these are our top priorities, then it sounds like our vision for C++23 is building foundations and addressing long standing issues.
  - Jens commented that such a vision fits in well with C++23 in general.
- Tom stated that the next telecon will be held on 5/27.

## May 27th, 2020

### Draft agenda:

- D1949R4: C++ Identifier Syntax using Unicode Standard Annex 31
  - Review updates since the April 22nd review.
- Discuss terminology updates to strive for in C++23
- [P1859R0: Standard terminology character sets and encodings](#)
- Establish priorities for terms to address.
- Establish a methodology for drafting wording updates.

### Attendees:

- Corentin Jabot
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Mark Zeren
- Martinho Fernandes
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann
- Zach Laine

### Meeting summary:

- D1949R4: C++ Identifier Syntax using Unicode Standard Annex 31
  - [Draft revision under discussion](#)
  - Steve summarized the changes since R3 and some additional perspectives on the history.
    - The most significant change is new wording provided by Jens.
    - This wording better matches the [UAX #31](#) design.
    - The wording changes introduce new *identifier-start* and *identifier-continue* grammar terms.
    - More motivation, history, and other editorial changes have been added.
    - Per recent discussion on the SG16 mailing list, Steve noted that `XID_Start` and `XID_Continue` were made stable some time after Unicode 5.2 and C++11.
    - At the time that C++11 was standardized, ISO recommendations were to define allowable identifiers as ranges of code points.
    - The C++11 design was based on the "Alternate Identifier Syntax" specified in the Unicode 5.2 version of UAX #31 and that was a reasonable choice at the time.
    - [ Editor's note: "Alternate Identifier Syntax" was renamed to "Immutable Identifiers" in Unicode 9. ]
    - Checking for non-NFC strings is significantly faster than actually normalizing; there are short cuts supported by Unicode.

- Zach provided some details regarding the NFC checking algorithm he implemented in Boost.Text. It was inspired by ICU and consists of four checking levels; the first of which is very fast and the rest are progressively slower to handle different edge cases.
- PBrett opined that we're all probably pretty comfortable with implementability of the proposal.
- Zach agreed, but noted that we need to be able to explain the complexity to EWG.
- Steve noted that the real question from implementors is about having to compare against the `XID_Start` and `XID_Continue` classes; fortunately we have implementation experience.
- Jens responded that implementation experience is not necessarily convincing. For example, EDG implemented support for C++98 exported templates, but that was an ill-designed feature. Describing an algorithm would be more useful.
- Steve noted that the paper contains links to [UAX #15](#) and the algorithms that it describes for detecting normalization form.
- PBrett commented that experience in other languages helps to illustrate viability.
- Steve stated that it is worth noting that the Unicode character database is not needed for implementation purposes.
- Tom asked if that is stated in the paper.
- Steve replied that it isn't.
- Tom asked Steve if we would be willing to add that.
- Steve replied, will do.
- Zack noted that the data needed for NFC normalization easily fits in a header and that the `XID_Start` and `XID_Continue` ranges are smaller.
- Steve noted that all of the needed data is listed in a verbose form in an appendix of the paper.
- PBrett asked if there are examples of script specific identifiers that are allowed today that will cease to be valid.
- Steve responded that there are examples in UAX #31.
- Tom asked if the change to *pp-number* should perhaps be *pp-number identifier* instead of *pp-number identifier-continue* since the non-numeric portion corresponding to *ud-suffix* needs to be a valid identifier for declaration of a user-defined literal function.
- Jens replied that such a change would work, but we currently use a max munch approach that allows, for example, `1x1x1x` to be a valid *pp-number*.
- Hubert observed that better diagnostic messages could be produced with such a change.
- Jens responded that the proposed changes are a consequence of other changes in the paper and are not intended to change the lexing behavior of *pp-number*. Tom's suggested change would be an unnecessary design change.
- Tom suggested the currently proposed wording sounds like what we want then.
- Corentin noted that the *identifier-start* and *identifier-continue* productions both include *nondigit*, but *nondigit* is a subset of *universal-character-name*.
- Steve responded that *corresponds to \uxxxx* where *as nondigit selects characters from the basic source character set*.
- Corentin asked what the intent was in creating a new kind of preprocessing-token that, when matched, always means the program is ill-formed.
- Tom responded that Corentin missed the meeting where this was discussed and that the previous meeting notes may be helpful. Basically, this allows rejecting lone combining characters.
- PBrett suggested that Corentin's question may suggest that this is too subtle.
- Jens responded that there are two levels of interpretation here. The first is the lexer grammar and it is only concerned with munging characters. The second is the formation of tokens. The new production allows issuance of nice diagnostics.
- PBrett asked if the last two sentences in that paragraph (`[lex.pptoken]p2`) could be swapped as that would better match the order of the grammar productions.
- Jens replied that they could be.
- Jens noted that italics were missing for three instances of *universal-character-name* and that, in the plural form, the ending "s" should not be italicized.
- Jens further noted that italics were missing for the use of "identifier" in the new wording in `[lex.name]`; italics are needed because this is a reference to the grammar term.
- Hubert stated that some implementations of markup make it difficult to not italicize plural suffixes, but that it is sometimes possible by using a ZWJ.
- Jens suggested that Steve not worry about it if removing italics for the plural suffix is difficult.
- Corentin observed that the existing reference to ISO/IEC 10646 and the new reference to UAX #44 are for distinct publications that may be out of sync.
- Jens responded that the existing reference to ISO/IEC 10646 is undated. Implementors should therefore use the latest available thus implying a moving target. New ISO/IEC 10646 versions become available more frequently than ISO C++ releases. This then requires implementors to update to newer ISO/IEC 10646 revisions in between ISO C++ releases.
- Jens stated a preference for a dated ISO/IEC 10646 reference that is updated with each ISO C++ release.
- Tom asked if a dated reference would allow implementors to adopt newer versions of ISO/IEC 10646 than the corresponding ISO C++ release references.
- Jens replied that they shouldn't in their pedantic modes.
- Hubert agreed.
- Zach added that Jens preference matches the guidance provided by LWG for [P1868](#) and the reference to [UAX #29](#).
- Corentin reiterated his claim that the various references require version correspondence.
- Hubert responded that, with respect to the characters made available, the intersection of characters available in the various specifications is what would matter; other characters would be rejected.
- Hubert observed that the reference to [UAX #44](#) should actually be a reference to the `DerivedCoreProperties.txt` file from the Unicode character database.



- Jens agreed that the normative reference we actually need is to `DerivedCoreProperties.txt` since UAX #44 does not contain its contents.
- Martinho clarified that UAX #44 describes the semantics for the contents of `DerivedCoreProperties.txt`, but not its syntax.
- Tom stated that it sounds like we need a normative reference to both then.
- Steve added that the versions of UAX #44 and `DerivedCoreProperties.txt` must be consistent.
- PBrett returned to the subject of dated vs undated references. The current reference to ISO/IEC 10646 is undated and these other references must match since they are dependent references on the version of ISO/IEC 10646.
- Tom asked if that implies that this paper must change the reference to ISO/IEC 10646 to a dated reference.
- Hubert responded that we can deal with that separately.
- Jens asked if a dated reference for UAX #44 and `DerivedCoreProperties.txt` is needed for this paper.
- Hubert responded that ISO/IEC 10646 version 5 does refer to Unicode character database files without reference to UAX #44.
- Steve noted that means transitive references exist.
- Hubert agreed, but noted that transitive references don't exist for all of the references needed.
- Corentin asked Steve which Unicode version the source of the XID data in the paper came from.
- Steve responded that it was probably Unicode 12.
- PBrett stated that, if we're going to pin down any one of these references, then we must pin down all of them. Otherwise, the correspondence doesn't make sense.
- Zach opined that these new references don't need to be in sync with ISO/IEC 10646, but that it would be nice if they were. We only need the normalization algorithm and XID data for this paper.
- Martinho asserted that the base line will be whatever is available at publication time.
- Hubert observed that we seem to have distinct needs. As far as UAX #31 is concerned, since it is only needed to satisfy references in the new informative annex, a dated reference should be used for it.
- Steve added that, since it is informative, the reference to UAX #31 is only needed in the bibliography.
- Hubert continued stating that, for UAX #44 and `DerivedCoreProperties.txt`, that a dated match to ISO/IEC 10646 is unnecessary and would be challenging for reasons of timing; ISO/IEC 10646 is currently in DIS status.
- Hubert summarized; the reference to UAX #31 should be dated, and the references to UAX #44 and `DerivedCoreProperties.txt` should be undated.
- Steve noted that leaving them undated might be helpful for applying these changes as a defect report for prior standards.
- Jens asked Hubert to confirm that a normative reference to `DerivedCoreProperties.txt` is required.
- Hubert confirmed that it is.
- Jens asked Steve to please add such a normative reference.
- Jens asked Hubert to confirm his opinion that the references to UAX #44 and `DerivedCoreProperties.txt` should be undated.
- Hubert confirmed and added that adding a date now would be counterproductive since there will be new publications of them before the next ISO C++ publication.
- Martinho provided a link for an undated reference to `DerivedCoreProperties.txt`.
- Tom reported being unable to find the wording updates to add UAX #31 to the bibliography.
- Steve reported that there had been a markdown issue that has since been fixed. The reference can be found by searching for ":::add".
- Hubert suggested that the reference to `DerivedCoreProperties.txt` should specify "as interpreted by UAX #44".
- [ Editor's note: [later discussion on the SG16 mailing list](#) proposed "The character classes XID\_Start and XID\_Continue are Derived Core Properties as described by UAX #44". ]
- Tom confirmed an intent to poll forwarding the paper to EWG with the discussed changes and asked for volunteers to validate that the updates are consistent with the discussion.
- Zach and Jens agreed to do so.
- **Poll: D1949R4: Forward to EWG with changes as discussed pending validation that updates reflect SG16 intent**
  - **Attendees: 11**
  - **No objection to unanimous consent.**
- Tom confirmed that the next meeting will be June 10th and that the topic will be terminology.